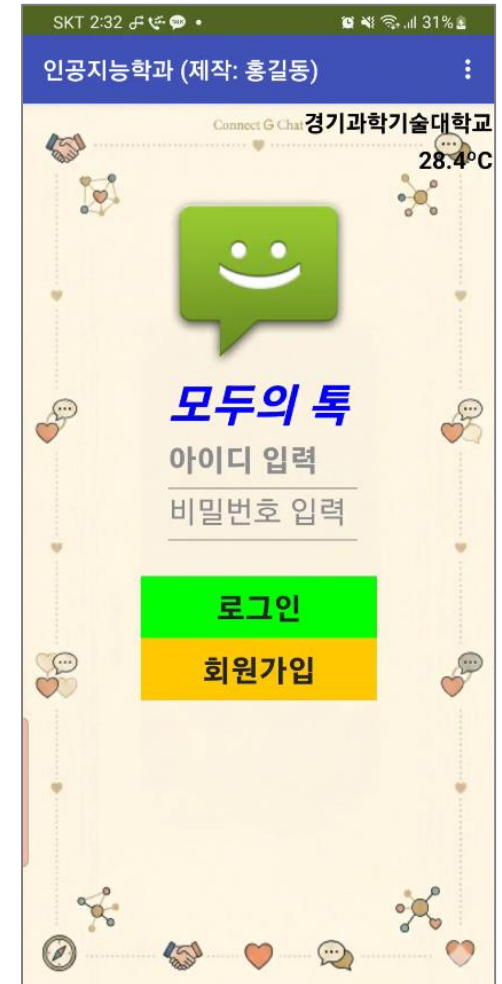
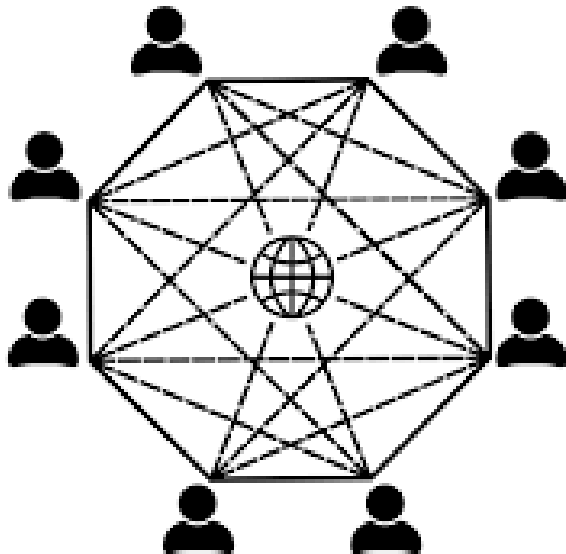
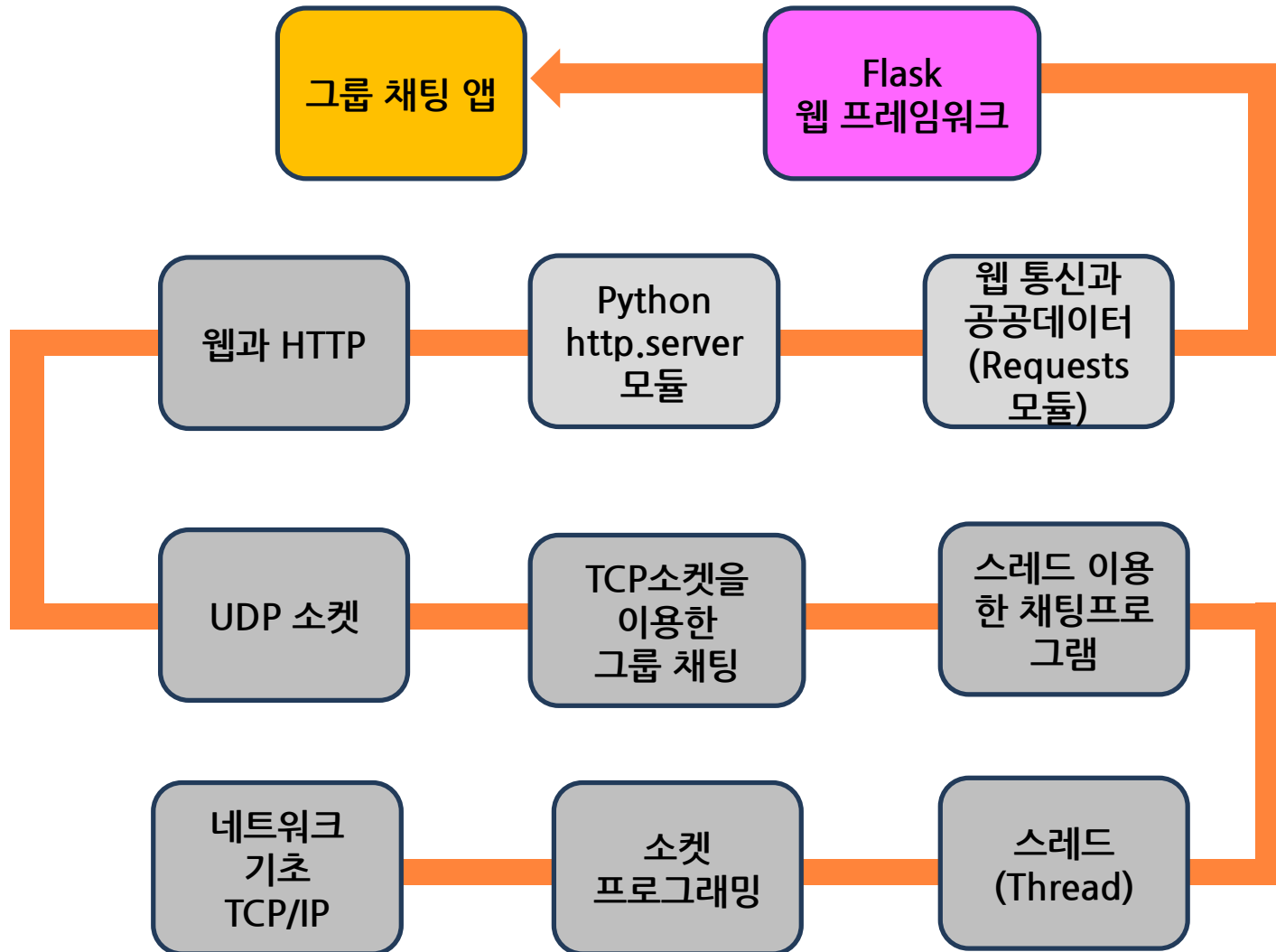


네트워크설계

14주차 - 그룹 채팅 앱



14주차는 과목 프로젝트인 '그룹 채팅 앱'을 제작해 봅시다

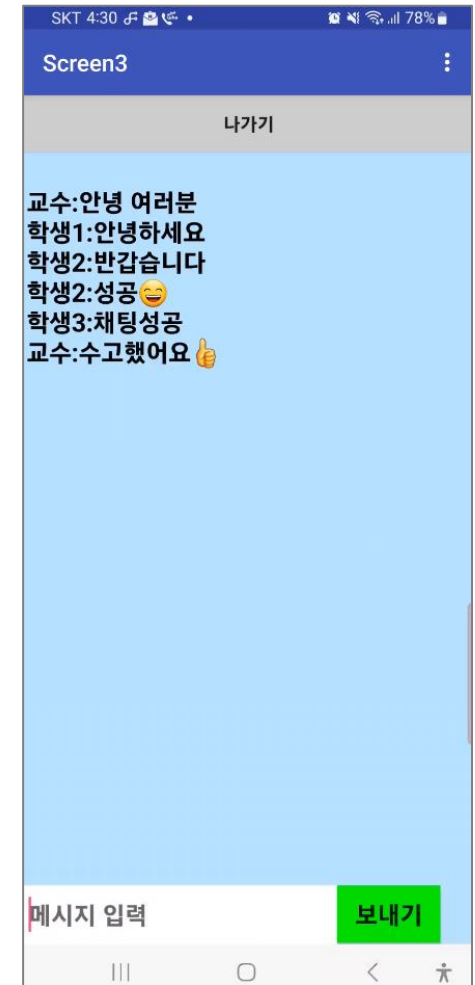
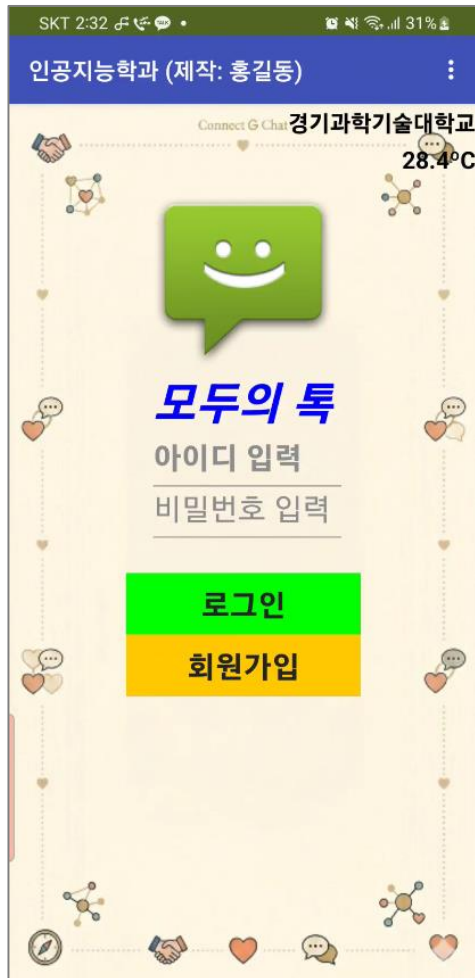


앱 제작 실습 병행

1. [그룹 채팅 앱] 제작 가이드

■ 우리 수업의 마무리는 [그룹채팅 앱] 제작하기

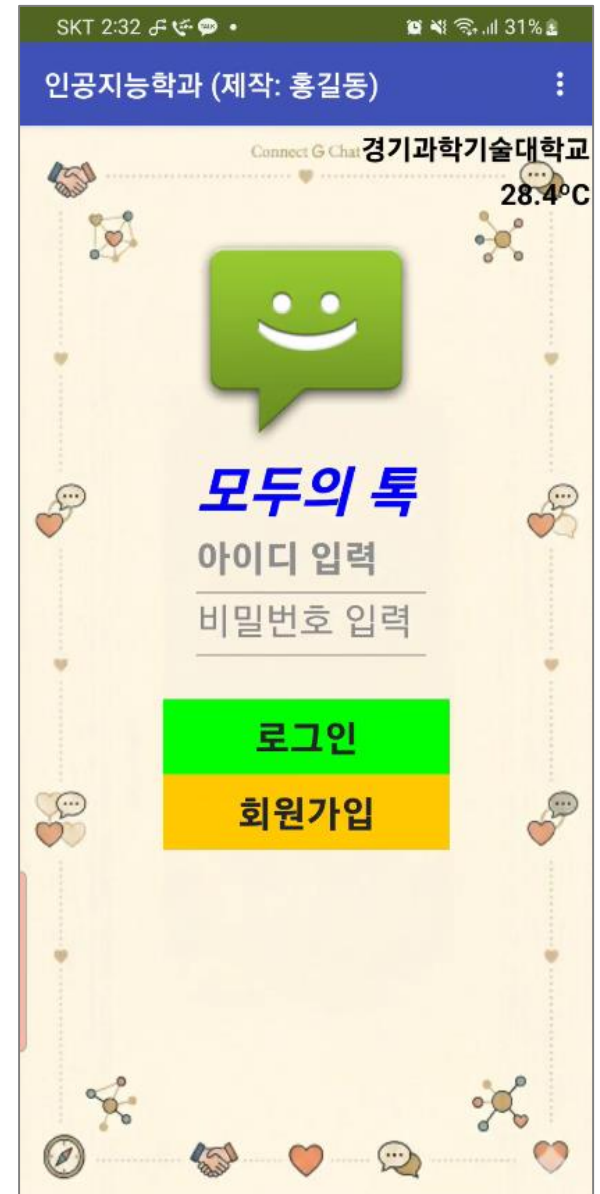
- 그 동안 수업한 앱 인벤터 앱 제작과 네트워크 지식을 활용하여 웹 기반에서 작동하는 우리 학과만의 [그룹채팅 앱]을 제작해 봅시다.



1. [그룹 채팅 앱] 제작 가이드

■ [그룹 채팅 앱] 제작 후 최종과제로 제출

- [그룹 채팅 앱]은 3단계 평가 (과제점수에 반영)
 - A : 제시된 기능 정상 작동 시
 - B : 제출은 되었으나 일부 기능 미 작동
 - C : 미 제출 시
- 앱 디자인은 자유이나 제시된 기능은 모두 작동되어야 함
- 앱은 3개의 스크린으로 구성(초기화면, 회원가입, 그룹채팅)
- 앱 주요 기능
 - 1) 학교주변 기온정보 (웹 파싱) - 12주차 정왕동 날씨 앱 활용
 - 2) 회원가입 기능
 - 3) 로그인 기능
 - 4) 그룹채팅 기능

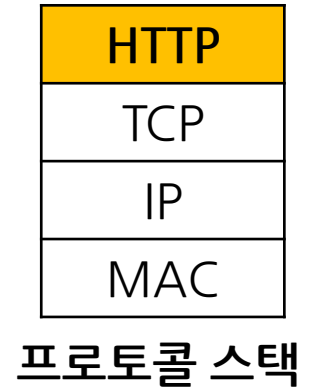


1. [그룹 채팅 앱] 제작 가이드

■ [그룹 채팅 앱] 구성도



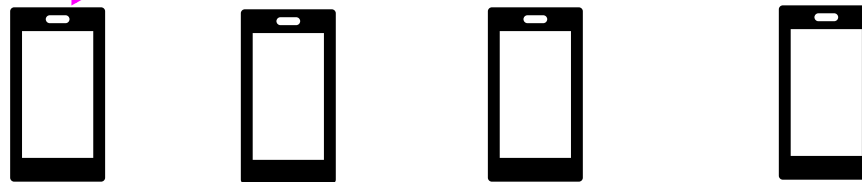
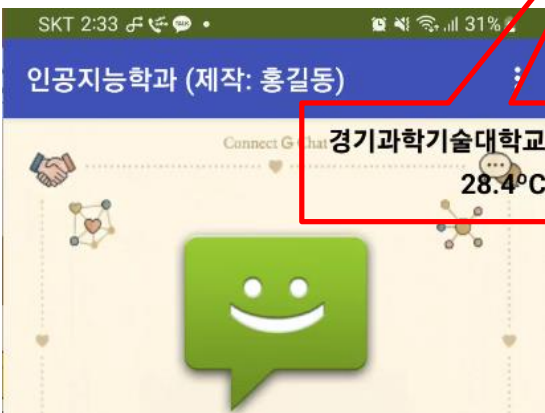
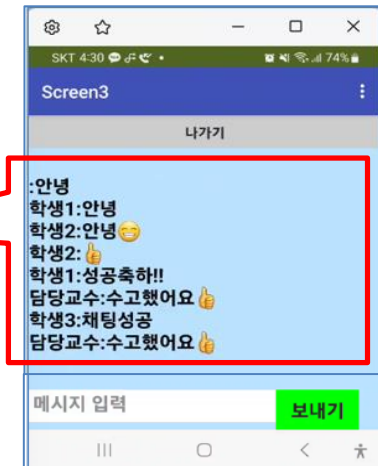
파이어베이스 구글 Cloud DB



정왕동
날씨 파싱
(11주차 실습)

DB 저장하기/가져오기

웹 통신(HTTP)



1. [그룹 채팅 앱] 제작가이드 - 파이어베이스 DB 만드는 법(참조만)

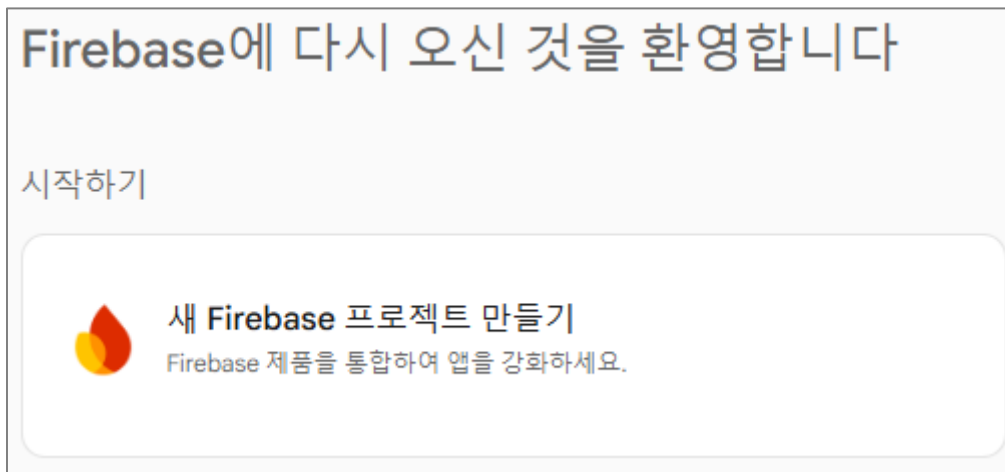
■ 파이어베이스 DB 만들기 (※ 참고만 하세요, DB를 만들 필요는 없습니다)

- 수업에 사용할 DB는 준비되어 있고 DB URL은 CTL사이트에 공지되어 있습니다

- 파이어베이스는 리얼타임 데이터베이스 제공 업체로, 구글에 인수되면서 통합 앱 플랫폼으로 확장

<https://firebase.google.com>

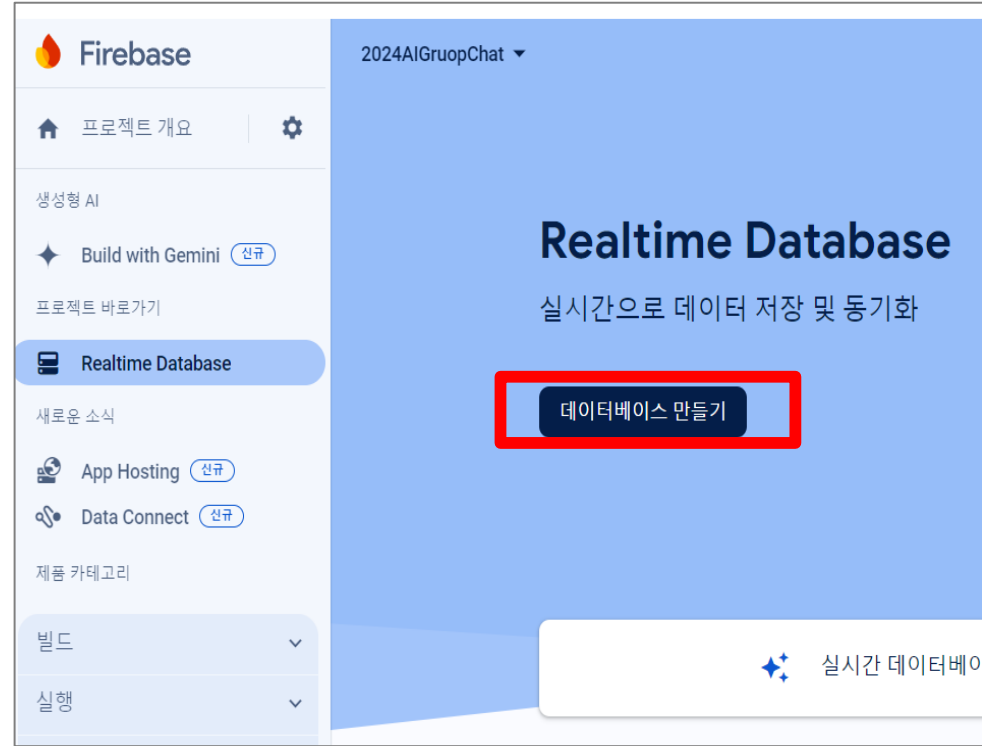
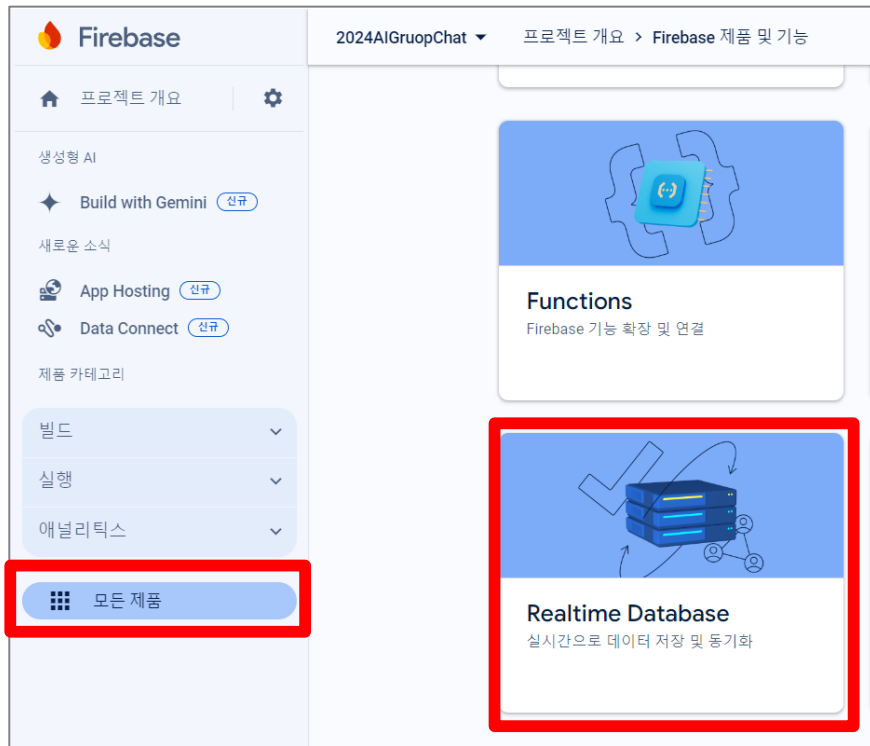
- 홈페이지 >> 시작하기 >> 새 프로젝트 만들기 >> 프로젝트 이름입력



1. [그룹 채팅 앱] 제작가이드 - 파이어베이스 DB 만드는 법(참조만)

■ 파이어베이스 DB 만들기 (※ 참고만 하세요, DB를 만들 필요는 없습니다)

- Realtime Database >> 데이터베이스 만들기



1. [그룹 채팅 앱] 제작가이드 - 파이어베이스 DB 만드는 법(참조만)

■ 파이어베이스 DB 만들기

데이터베이스 설정

1 데이터베이스 옵션

2 보안 규칙

위치 설정은 실시간 데이터베이스 데이터가 저장되는 위치입니다.

미국(us-central1)

취소

다음

- 테스트 모드로 30일간 사용가능

데이터베이스 설정

1 데이터베이스 옵션

2 보안 규칙

데이터 구조를 정의한 후 데이터의 보안을 강화하는 규칙을 작성해야 합니다.
[자세히 알아보기](#)

☐ 잠금 모드로 시작

데이터는 기본적으로 비공개됩니다. 클라이언트 읽기/쓰기 액세스 권한은 보안 규칙에서 지정한 대로만 부여됩니다.

☒ 테스트 모드에서 시작

빠른 설정을 위해 기본적으로 데이터가 공개됩니다. 하지만 30일 내에 보안 규칙을 업데이트하여 장기적 클라이언트 읽기/쓰기 액세스 권한을 사용 설정해야 합니다.

```
{
  "rules": {
    ".read": "now < 1687878000000", // 2023-6-28
    ".write": "now < 1687878000000", // 2023-6-28
  }
}
```

!

테스트 모드의 기본 보안 규칙에서는 데이터베이스 참조 사용자는 누구나 향후 30일 동안 데이터베이스의 모든 데이터를 보고 수정하며 삭제할 수 있습니다.

취소

사용 설정

1. [그룹 채팅 앱] 제작가이드 - 파이어베이스 DB 사용법

■ 파이어베이스 DB 만들기

- 파이어베이스 DB 만들고 앱 인벤터 앱에서 사용하기

The screenshot shows the Firebase console interface. On the left, the 'Firestore' sidebar is visible with options like '제품 검색', '프로젝트 개요', '설정', '새로운 소식', 'SQL Connect', 'Phone Verification', '프로젝트 바로가기', 'Realtime Database', '제품 카테고리', '데이터베이스 및 스토리지', and '보안'. The main area displays the 'Realtime Database' for the project '2026NetworkDesignProject'. A red box highlights the '데이터' (Data) tab, showing a tree structure with 'chat' and 'user' nodes. The 'chat' node contains a message, and the 'user' node contains a user ID. A red box also highlights the 'URL' field in the '데이터' tab, which contains the URL 'https://networkdesignproject-f90ba-default-rtdb.firebaseio.com/'. A red arrow points from this URL to the '파이어베이스 URL' field in the '속성' (Properties) tab on the right, which also contains the same URL. A red box highlights the '파이어베이스DB1' component in the '컴포넌트' (Components) list on the right. A red box also highlights the '파이어베이스DB1' component in the '속성' (Properties) tab on the right.

2026NetworkDesignProject ▼

Realtime Database

데이터 규칙 백업 사용량 Extens

결제 사기나 피싱과 같은 악용으로부터 Realtime Database 리소스를 보호하세요.

<https://networkdesignproject-f90ba-default-rtdb.firebaseio.com>

<https://networkdesignproject-f90ba-default-rtdb.firebaseio.com/>

- chat
 - msg: ""\n교수:안녕하세요 여러분\n교수:네트워크설계 단톡방에 오신것을 환영합니다""
- user
 - 교수: ""1234""

The screenshot shows the '속성' (Properties) tab for the '파이어베이스DB1' component. The '파이어베이스URL' field is highlighted with a red box and contains the URL 'https://groupchatai-bb2dc-d.firebaseio.com'. A red arrow points from this URL to the '파이어베이스 URL' field in the '데이터' tab on the left, which also contains the same URL. A red box highlights the '파이어베이스DB1' component in the '컴포넌트' (Components) list on the left.

속성

파이어베이스DB1

파이어베이스토큰

파이어베이스URL

<https://groupchatai-bb2dc-d.firebaseio.com>

☐ 기본값 사용

지속여부

☐

프로젝트버킷

user

파이어베이스 URL

※DB URL은 공지사항 참조

1. [그룹 채팅 앱] 제작가이드 - 파이어베이스 DB 사용법

■ 파이어베이스 DB 만들기

- 그룹 채팅앱에 사용될 DB 는 두개의 버킷으로 구성
 - **chat** 버킷에는 채팅 메시지 저장 (Screen1과 Screen3에 사용)
 - **user** 버킷에는 아이디와 비밀번호 저장 (Screen2에 사용)

파이어베이스DB1

▼ Behavior

파이어베이스토큰 ?
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1b290eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1b290eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9

파이어베이스URL ?
<https://networkdesignproject-f90ba-default-rtdb.firebaseio.com/>

☐ 기본값 사용

지속여부 ?
☐

프로젝트버킷 ?
user

 <https://networkdesignproject-f90ba-default-rtdb.firebaseio.com/>

<https://networkdesignproject-f90ba-default-rtdb.firebaseio.com/>

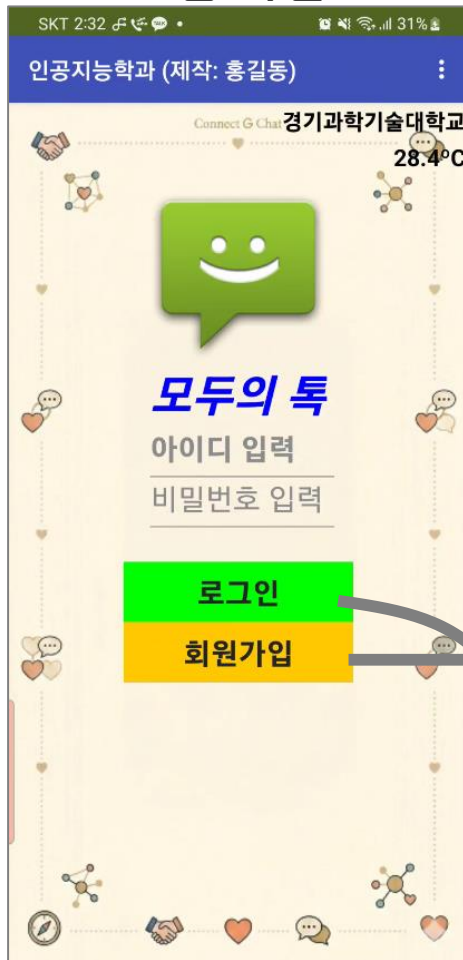
- ▼ **chat** **채팅 메시지 저장**
 - msg: ""\n교수:안녕하세요 여러분\n교수:네트워크설계 단톡방에 오신것을 환영합니다""
- ▼ **user** **아이디와 비밀번호**
 - 교수: ""1234""

1. [그룹 채팅 앱] 제작 가이드 - 앱 설명

■ 그룹채팅 앱 화면구성

- 앱은 총 3개의 Screen으로 구성

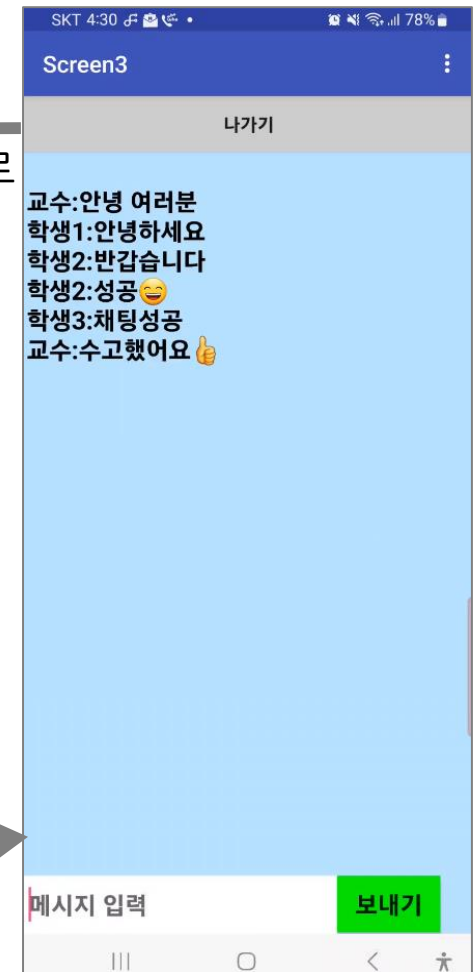
Screen1 (날씨+로그인)
홈 화면



Screen2 (회원가입)



Screen3 (그룹채팅 방)



[나가기]시 초기화면으로

[회원가입] 후
초기화면

회원가입

아이디 입력

비밀번호 입력

회원가입

취소하기

[로그인]시 채팅화면으로

1. [그룹 채팅 앱] 제작 가이드 - 앱 설명

■ Screen1 : 초기화면(날씨 파싱 + 로그인)

- 디자인은 자유입니다. 톡 이름도 자유입니다. 단, 제시된 기능은 정상동작해야 함

① 정왕동 날씨 파싱
(11주차 코딩블럭
재활용합니다!)

② 아이디와
비밀번호 입력 후
[로그인] 기능
→ Screen3 채팅화면
으로 이동



GET 메소드
request



response



- 파싱(Parsing): 웹 페이지에서 원하는 데이터를 추출하여 가공하는 것
- ③ [회원가입] 클릭 시
Screen2 회원가입 화면으로 이동

1. [그룹 채팅 앱] 제작 가이드 - 앱 설명

■ Screen2: 회원가입 화면

- 디자인은 자유입니다. 단, 제시된 기능은 정상동작해야 함

① 아이디와 비밀번호로
[회원가입] 및 Screen1으
로 이동

② [취소하기] 버튼 클릭 시
Screen1으로 이동

Screen2

회원가입

아이디 입력

비밀번호 입력

회원가입

취소하기

파이어베이스
구글 Cloud DB
(user, chat 버킷)



DB의 user 버킷에
아이디와 비밀번호 저장



1. [그룹 채팅 앱] 제작 가이드 - 앱 설명

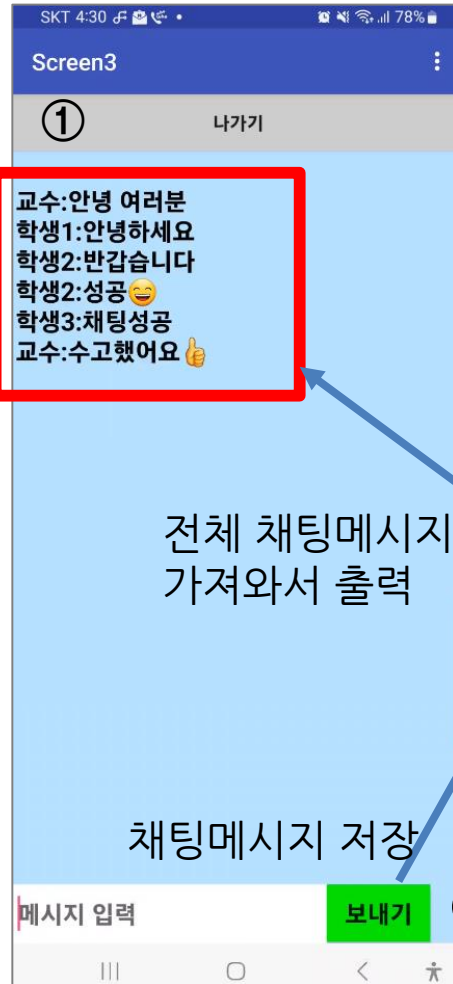
■ Screen3: 그룹 채팅 화면

- 디자인은 자유입니다. 단, 제시된 기능은 정상동작해야 함

① [나가기] 버튼
클릭 시 Screen1으로
이동

②

② 주기적으로(1초간격)
으로 DB의 채팅메시지
가져와서 출력하기



전체 채팅메시지
가져와서 출력

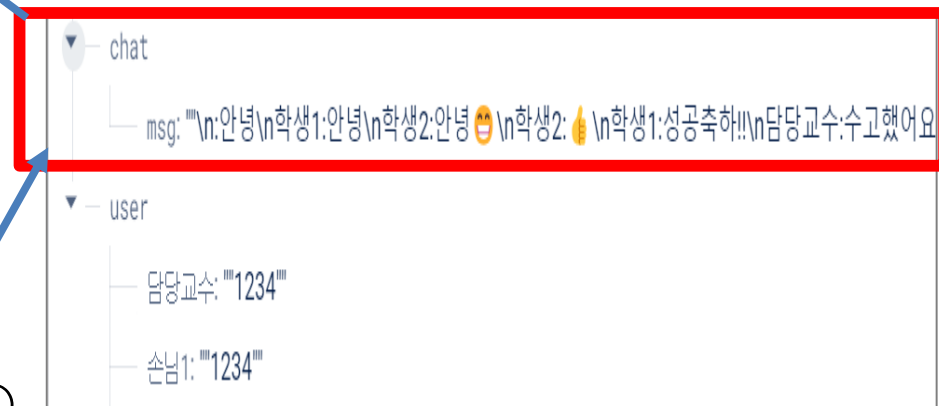
채팅메시지 저장

③ [보내기] 버튼 클릭시
채팅메시지 DB로 전송하
기

파이어베이스
구글 Cloud DB
(user, chat 버킷)



채팅 메시지는
DB의 chat 버킷에 저장



1. [그룹 채팅 앱] 제작가이드 - 11주차 기상청 파싱앱 백팩에 넣기

■ 11주차에 제작한 기상청 기온정보 가져오기 앱의 코드를 백팩에 저장

- [그룹채팅 앱]에 정왕동 현재 기온을 표시하는 기능이 있음
- 이 부분은 11주차 작성한 코드를 백팩에 넣어서 재활용하기 바랍니다
- 11주차 앱에서 코드를 백팩에 넣고, 다른 프로젝트에서 꺼내 활용하면 됨

넣기

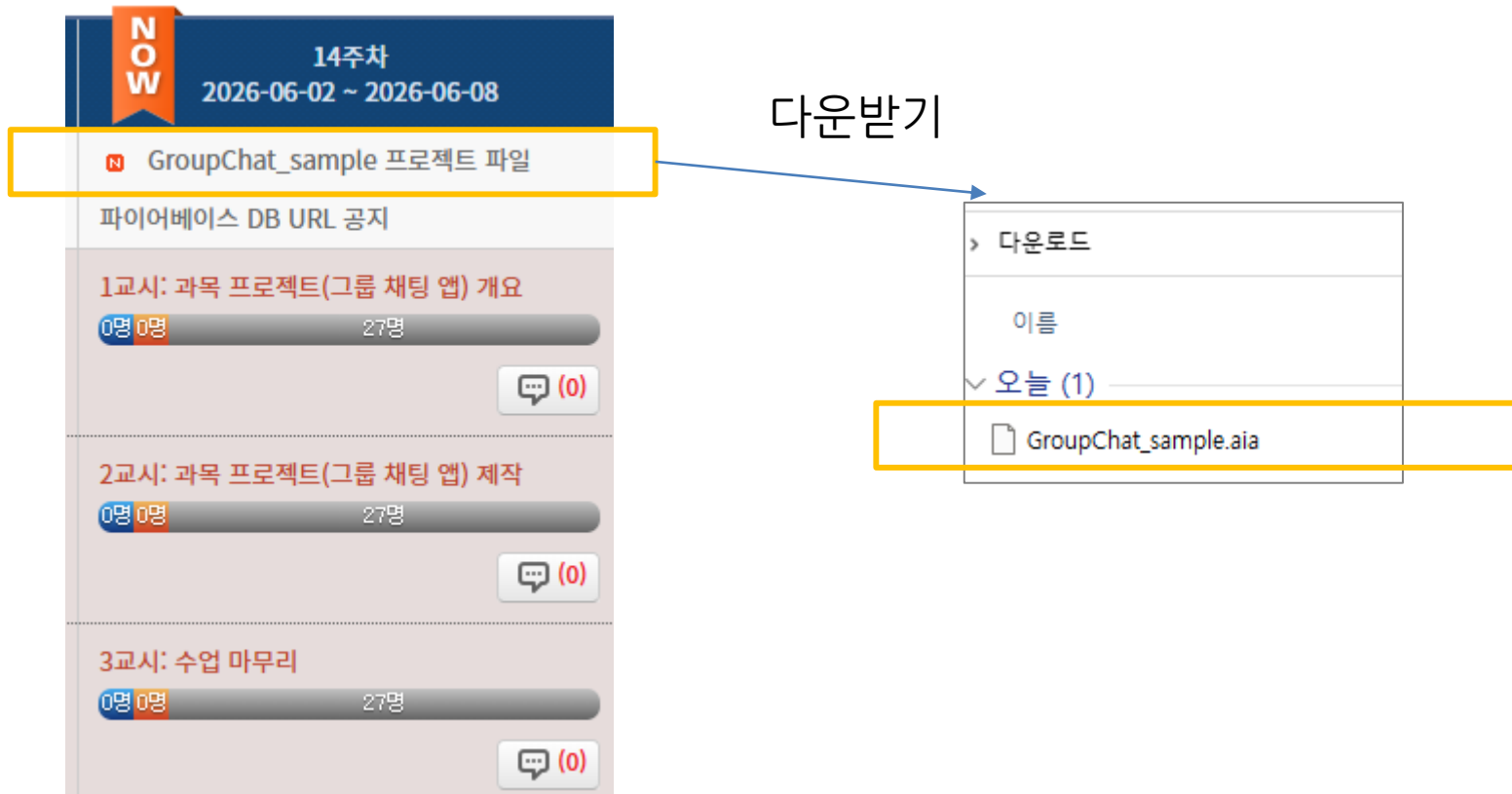
가져오기

다른 프로젝트에서 활용

The image shows a Scratch-like block-based programming environment. The code is organized into a 'when green flag clicked' event, followed by a 'when timer starts' loop. The loop contains several 'set variable to value' blocks for 'current_date', 'current_time', and 'current_temp'. It also includes 'call web API' blocks to fetch weather data from the Korea Meteorological Administration. A blue arrow labeled '넣기' (Put) points from the code to a backpack icon, and a red arrow labeled '가져오기' (Get) points from the backpack icon to the code. Below the backpack icon, the text '다른 프로젝트에서 활용' (Use in other projects) is displayed.

1. [그룹 채팅 앱] 제작가이드 - 공지사항의 sample 프로젝트 활용하기

- 공지사항에 sample 프로젝트를 올려놓았습니다
- 다운받아 내 컴퓨터에 저장하기



1. [그룹 채팅 앱] 제작가이드 - 공지사항의 sample 프로젝트 활용하기

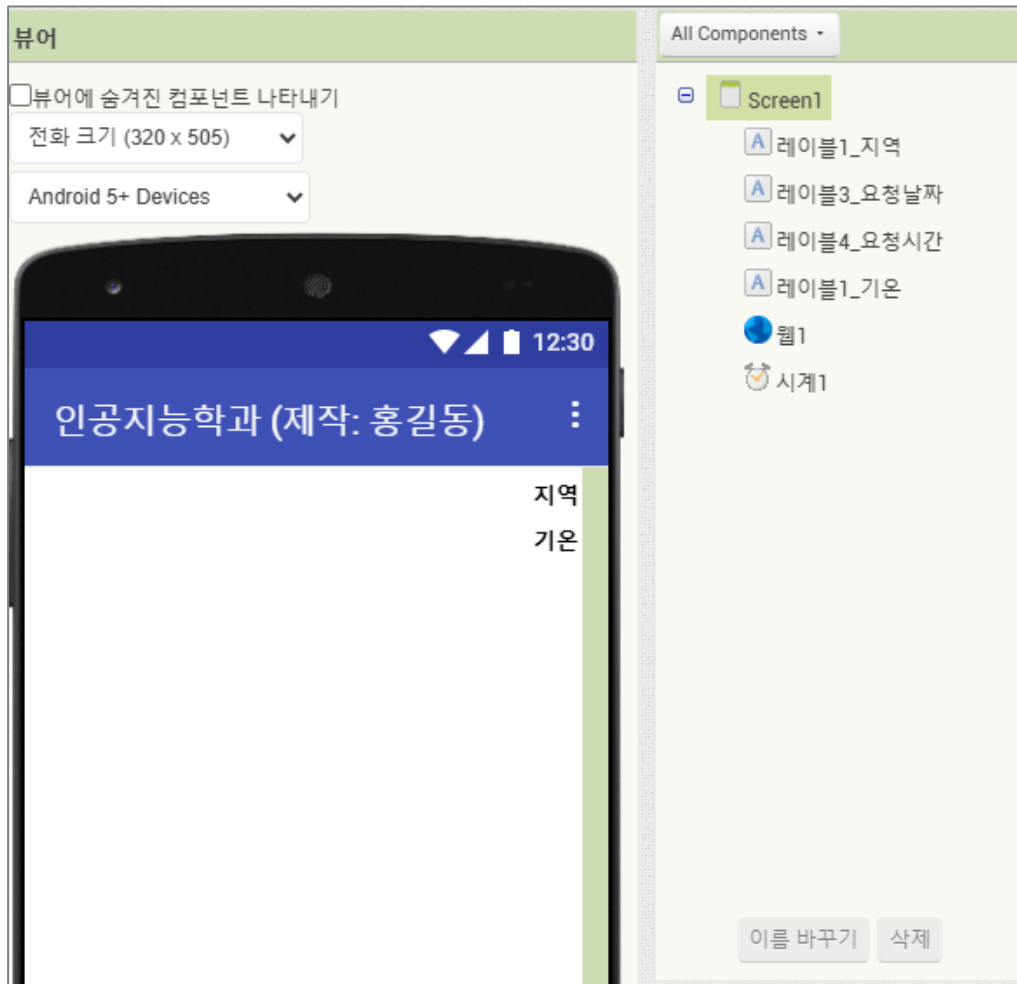
■ 앱 인벤터에 내 프로젝트로 저장하기

- 앱 인벤터 프로젝트 >> 내 컴퓨터에서 프로젝트(.aia) 가져오기

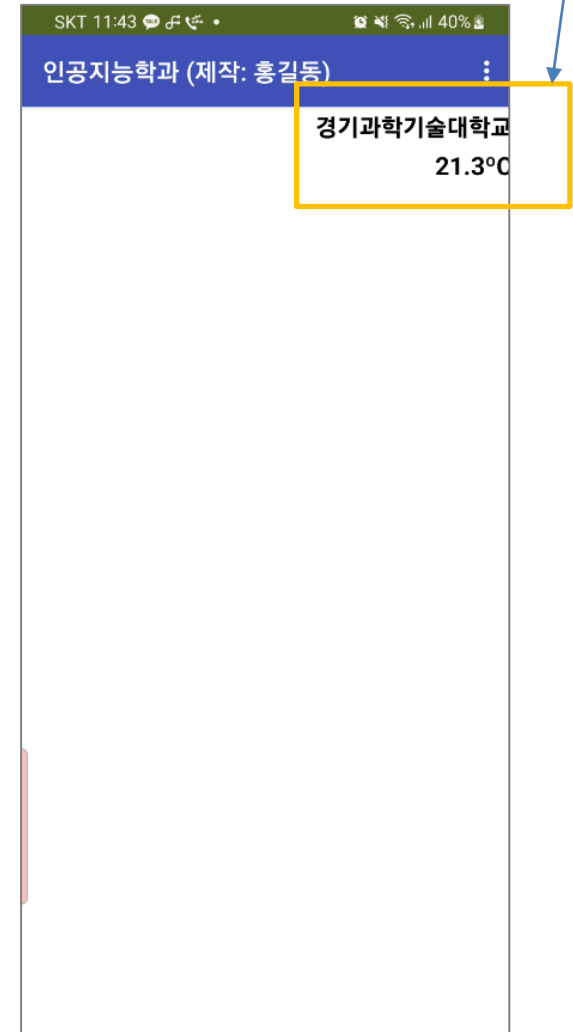


1. [그룹 채팅 앱] 제작가이드 - 공지사항의 sample 프로젝트 활용하기

- 앱 인벤터에 내 프로젝트로 저장하기
 - GroupChat_sample 프로젝트가 실행



기온정보가 정상적으로
파싱되는지 확인



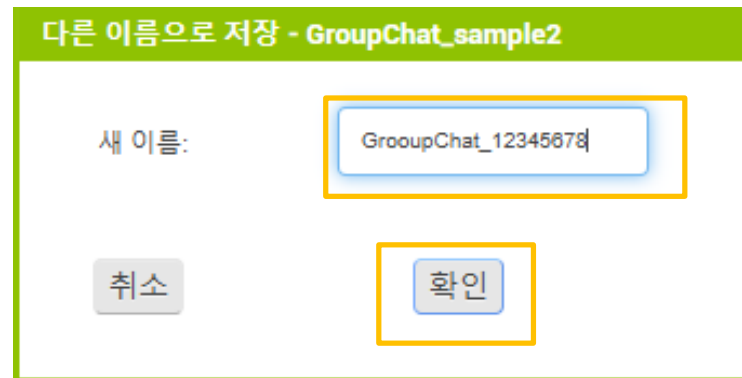
1. [그룹 채팅 앱] 제작가이드 - 공지사항의 sample 프로젝트 활용하기

■ 프로젝트 이름 바꾸기

- 프로젝트 >> 프로젝트 다른 이름으로 저장하기



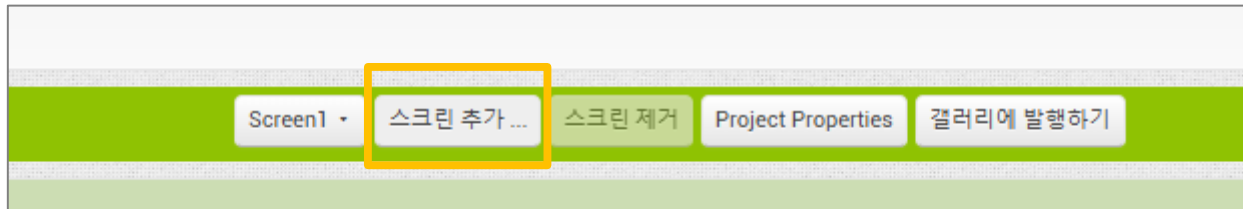
새 이름 : GroupChat_학번



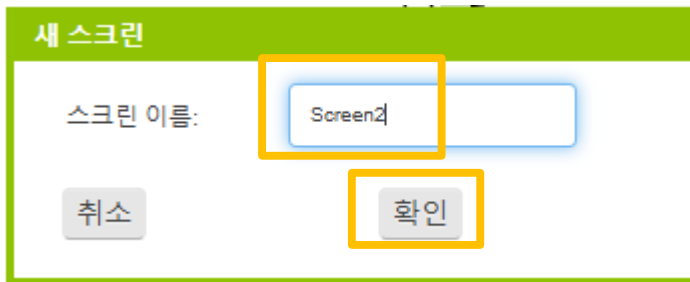
1. [그룹 채팅 앱] 제작가이드 - 공지사항의 sample 프로젝트 활용하기

■ Screen2, Screen3 추가하기

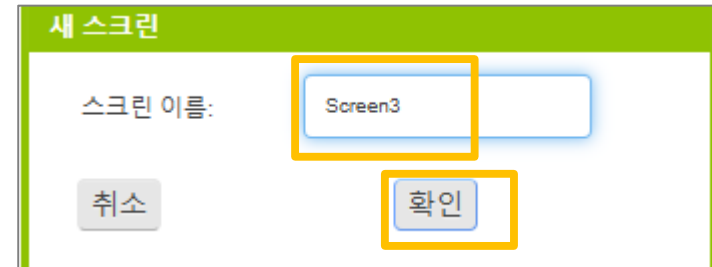
- Screen2, Screen3 추가하기



Screen2 추가



Screen3 추가



→ Screen1으로 돌아가서 디자인 시작하기

2. [그룹 채팅 앱] 제작 - Screen1 디자인

■ 그룹채팅 앱 제작하기 - Screen1 (날씨+로그인) 디자인

* 프로젝트이름- GroupChat



보이지 않는 컴포넌트



나머지 부분(노란색 파트) 디자인에 추가하기

	컴포넌트	이름	속성
	Screen	Screen1	제목 : 인공지능학과 - 000 (이름) 수평정렬-가운데 스크롤가능여부-체크 배경화면 - 여러분 자유로!
①	레이블	레이블_지역	글꼴굵게, 글꼴크기-15, 너비-100% 텍스트 - 지역, 텍스트정렬- 오른쪽
숨김	레이블	레이블_요청날짜	너비-부모요소, 텍스트 - 요청날짜, 텍스트정렬- 오른쪽, 보이기여부 - 해제
숨김	레이블	레이블_요청시간	너비-부모요소, 텍스트 - 요청시간, 텍스트정렬- 오른쪽, 보이기여부 - 해제
②	레이블	레이블_기온	글꼴굵게, 글꼴크기-15, 너비-100% 텍스트 - 기온, 텍스트정렬- 왼쪽
③	이미지	이미지1	여러분 자유로 디자인 하세요!
④	레이블	레이블_특이름	글꼴굵게, 글꼴크기- 40 텍스트 - 여러분 자유!!
⑤	텍스트박 스	텍스트박스1_ 사용자명	글꼴굵게, 글꼴크기- 20, 힌트-아이디 입력
⑥	암호텍 스트박스	암호텍스트박스1_ 비밀번호	글꼴굵게, 글꼴크기- 15, 힌트-비밀번호 입력
⑦	버튼	버튼1_로그인	배경색-자유, 글꼴굵게, 너비-50%, 모양-자유 , 텍스트-로그인
⑧	버튼	버튼1_회원가입	배경색-자유, 글꼴굵게, 너비-50%, 모양-자유 , 텍스트-회원가입

2. [그룹 채팅 앱] 제작 - Screen1 디자인

■ 그룹채팅 앱 제작하기 - Screen1 (날씨+로그인) 디자인



나머지 부분(노란색 파트) 디자인에 추가하기

	컴포넌트	이름	속성
⑨	웹	웹1	
⑩	알림	알림1	알림표시시간 - 짧게
⑪	파이어베이스DB	파이어베이스DB1	파이어베이스URL - https://networkdesignproject-f90ba-default-rtdb.firebaseio.com/ (CTL 공지사항에서 copy 하세요) 프로젝트버킷 - user
⑫	시계	시계1	

*파이어베이스DB: 팔레트 >> 실험실>> 파이어베이스 DB

* 시계 : 센서 >> 시계

2. [그룹 채팅 앱] 제작 - Screen1 코딩

■ 그룹채팅 앱 제작하기 - Screen1(날씨정보 파싱) 코딩

① 기상청으로부터 날씨정보 가져와서 파싱하기(1)

전역변수 만들기 APIkey 초기값

RW

전역변수 만들기 weather 초기값

전역변수 만들기 weather_data 초기값

전역변수 만들기 weather_temp 초기값

전역변수 만들기 current_date 초기값

전역변수 만들기 current_time 초기값

전역변수 만들기 base_date 초기값

전역변수 만들기 base_time 초기값

미리 코딩되어 있음

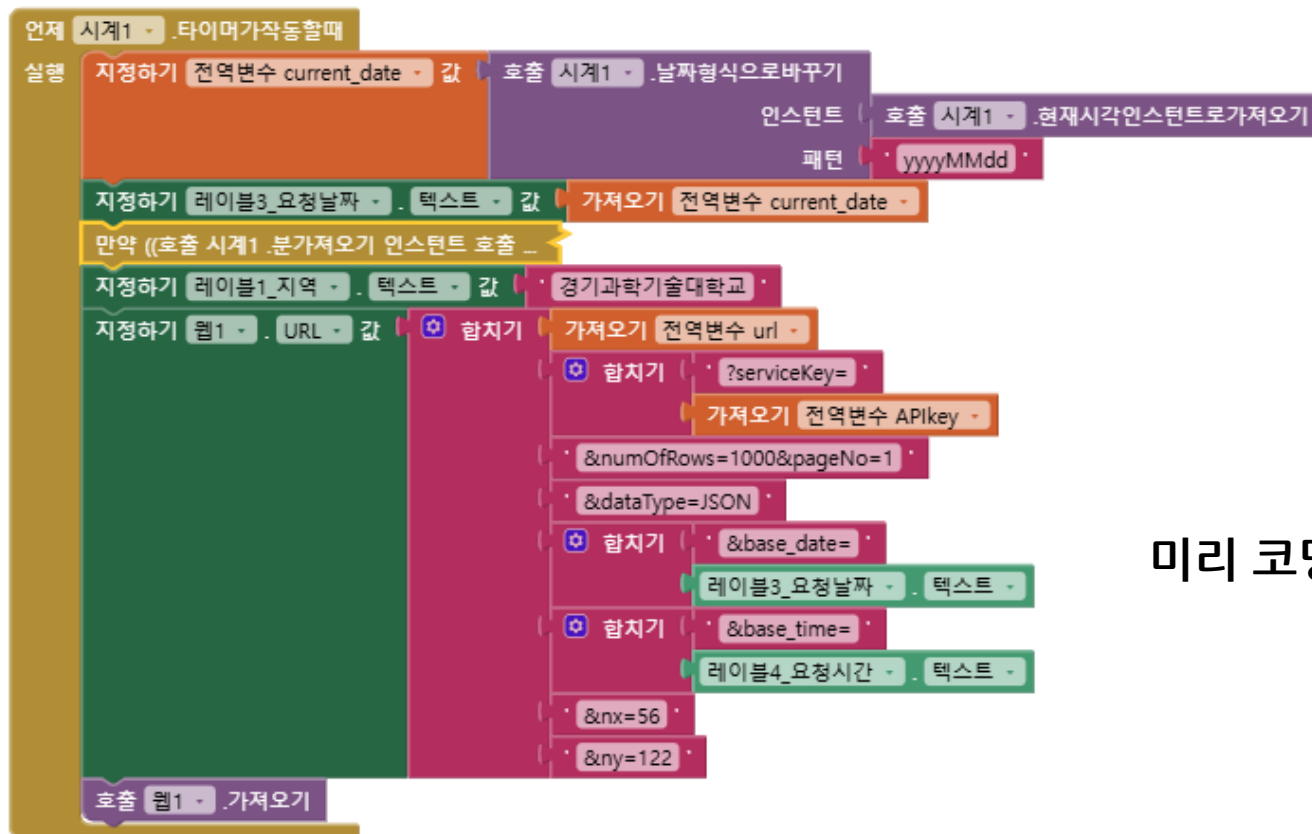
2. [그룹 채팅 앱] 제작 - Screen1 코딩

■ 그룹채팅 앱 제작하기 - Screen1(날씨정보 파싱) 코딩

① 기상청으로부터 날씨정보 가져와서 파싱하기(2)

※ [시계(타이머)] 컴포넌트의 역할:

- 일정 시간 간격으로 어떤 명령들을 실행하고자 할 때 사용
- 앱에서 5초마다 기상청 날씨누리 서버로부터 정보를 받아서 실시간 업데이트

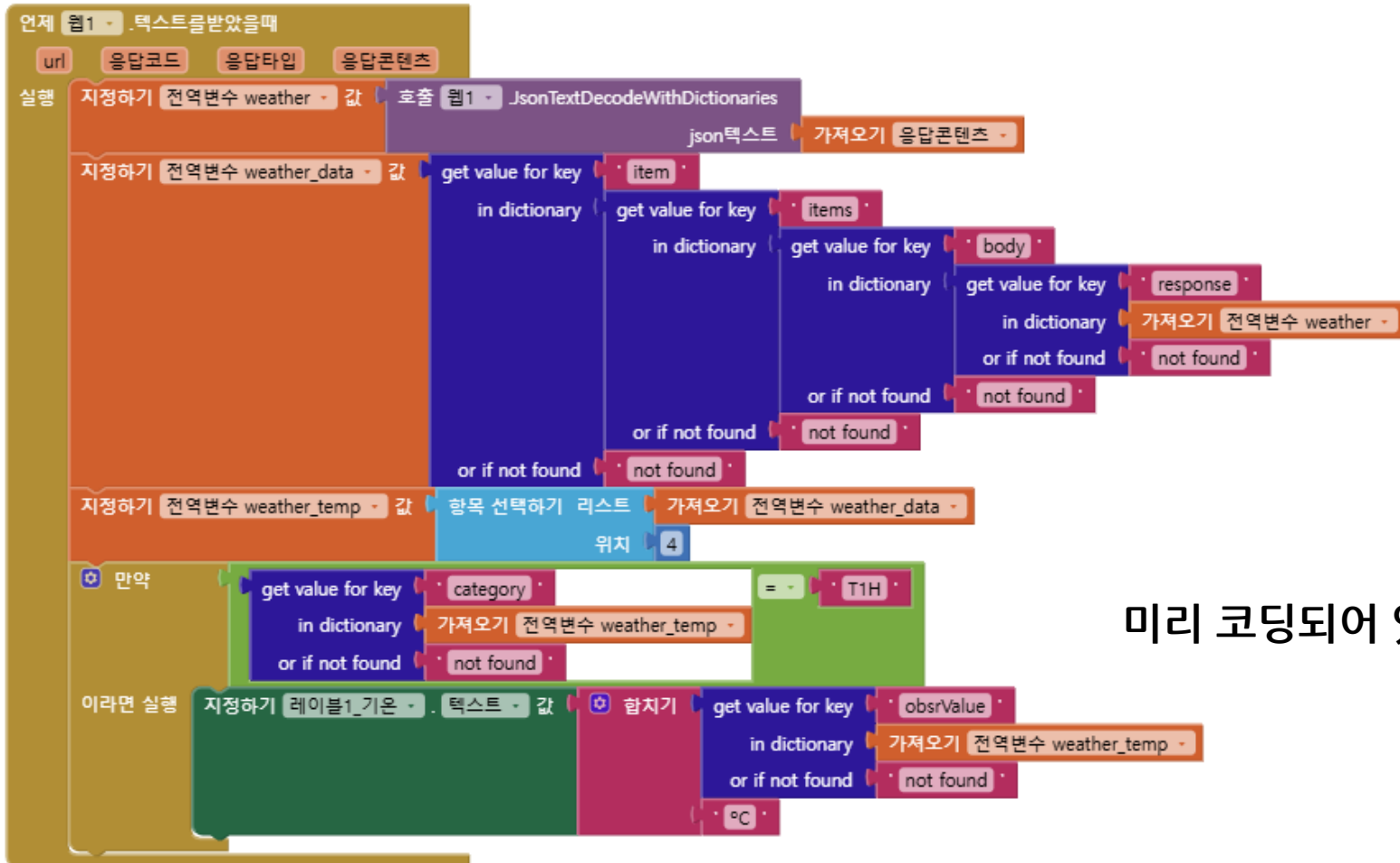


미리 코딩되어 있음

2. [그룹 채팅 앱] 제작 - Screen1 코딩

■ 그룹채팅 앱 제작하기 - Screen1(날씨정보 파싱) 코딩

① 기상청으로부터 날씨정보 가져와서 파싱하기(3)

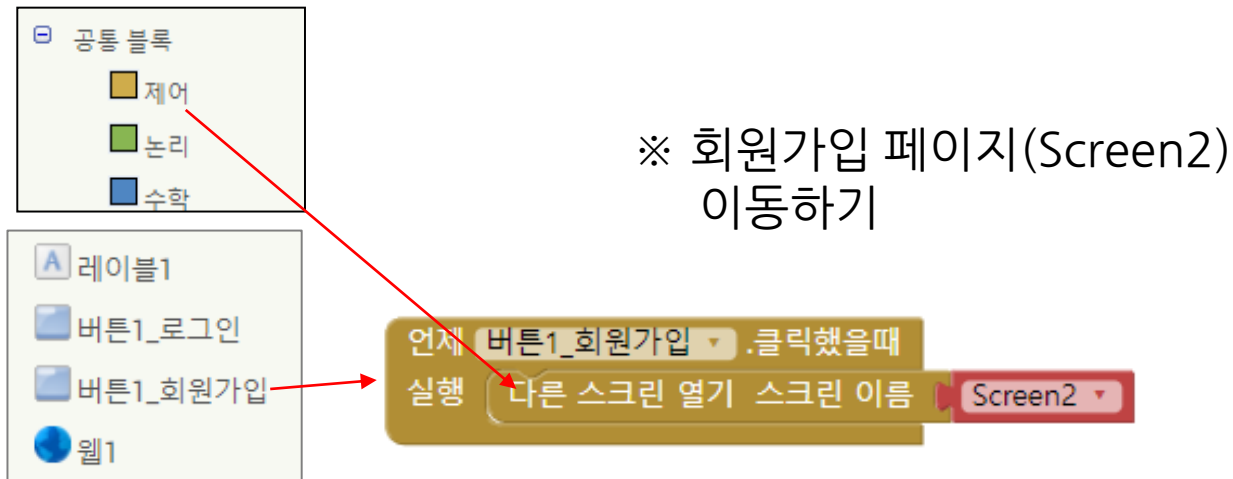


미리 코딩되어 있음

2. [그룹 채팅 앱] 제작 - Screen1 코딩

■ 그룹채팅 앱 제작하기 - Screen1(날씨정보 파싱) 코딩

② 회원가입 버튼 클릭 시 Screen2 페이지로 이동하기



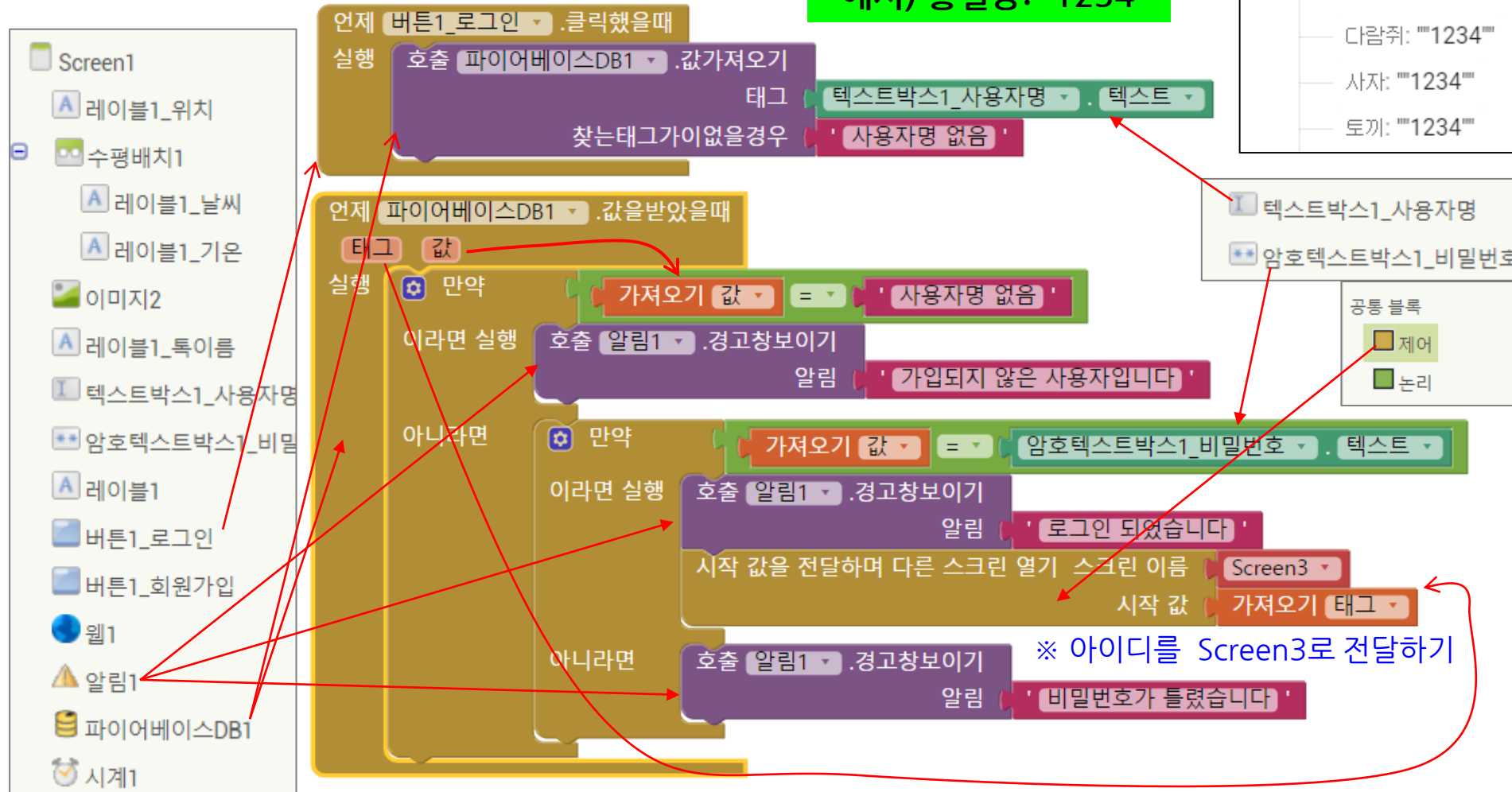
2. [그룹 채팅 앱] 제작 - Screen1 코딩

■ 그룹채팅 앱 제작하기 - Screen1(로그인) 코딩

③ 아이디(이름)과 비밀번호로 로그인 하기

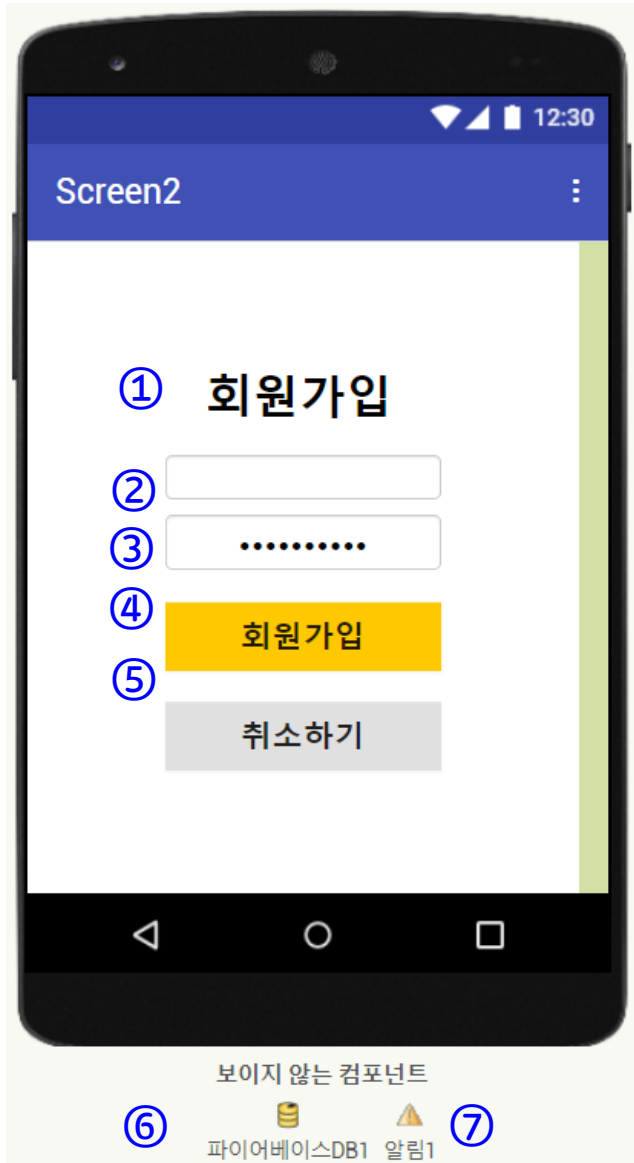
태그(아이디): 값
예시) 홍길동: "1234"

user	태그: 값
haha:	""1234""
hong:	""1234""
kim:	""1234""
lee:	""1234""
park:	""1234""
거북이:	""1234""
다람쥐:	""1234""
사자:	""1234""
토끼:	""1234""



3. [그룹 채팅 앱] 제작 - Screen2 디자인

■ 그룹채팅 앱 제작하기 - Screen2(회원가입) 디자인



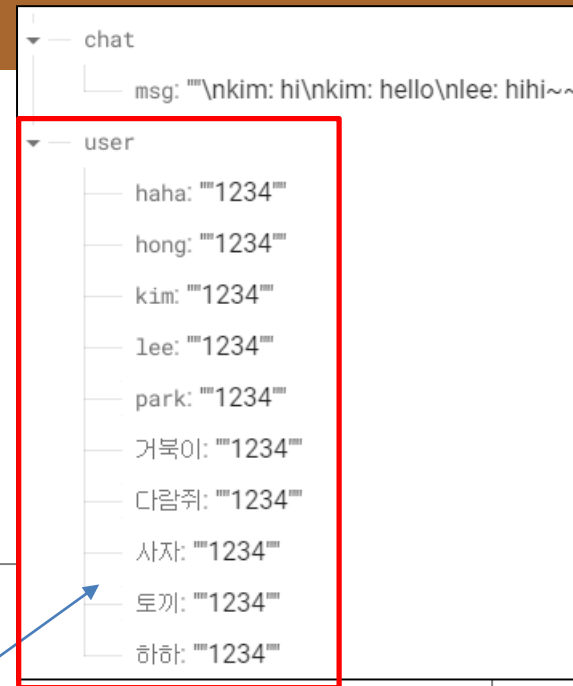
※ 레이블을 이용해 컴포넌트 사이에 여백을 적절히 디자인 하세요.

	컴포넌트	이름	속성
	Screen2	Screen2	수평정렬-가운데
①	레이블	레이블_회원가입	글꼴굵게, 글꼴크기-30 텍스트 - 회원가입, 텍스트정렬- 가운데
②	텍스트박스	텍스트박스1_아이디	글꼴굵게, 글꼴크기-20, 힌트-아이디를 입력, 텍스트정렬-가운데
③	암호텍스트 박스	암호텍스트박스1_비 밀번호	글꼴굵게, 글꼴크기-20, 힌트-비밀번호 입력, 텍스트정렬-가운데
④	버튼	버튼1_회원가입	배경색-자유, 글꼴굵게, 글꼴크기-20, 너비-50%, 텍스트-회원가입
⑤	버튼	버튼1_취소하기	배경색-자유, 글꼴굵게, 글꼴크기-20, 너비-50%, 텍스트-취소하기
⑥	파이어베이 스DB	파이어베이스DB1	파이어베이스URL - https://networkdesignproject-f90ba-default-rtdb.firebaseio.com/ 프로젝트버킷 - user
⑦	알림	알림1	알림표시시간 - 짧게

3. [그룹 채팅 앱] 제작 - Screen2 코딩

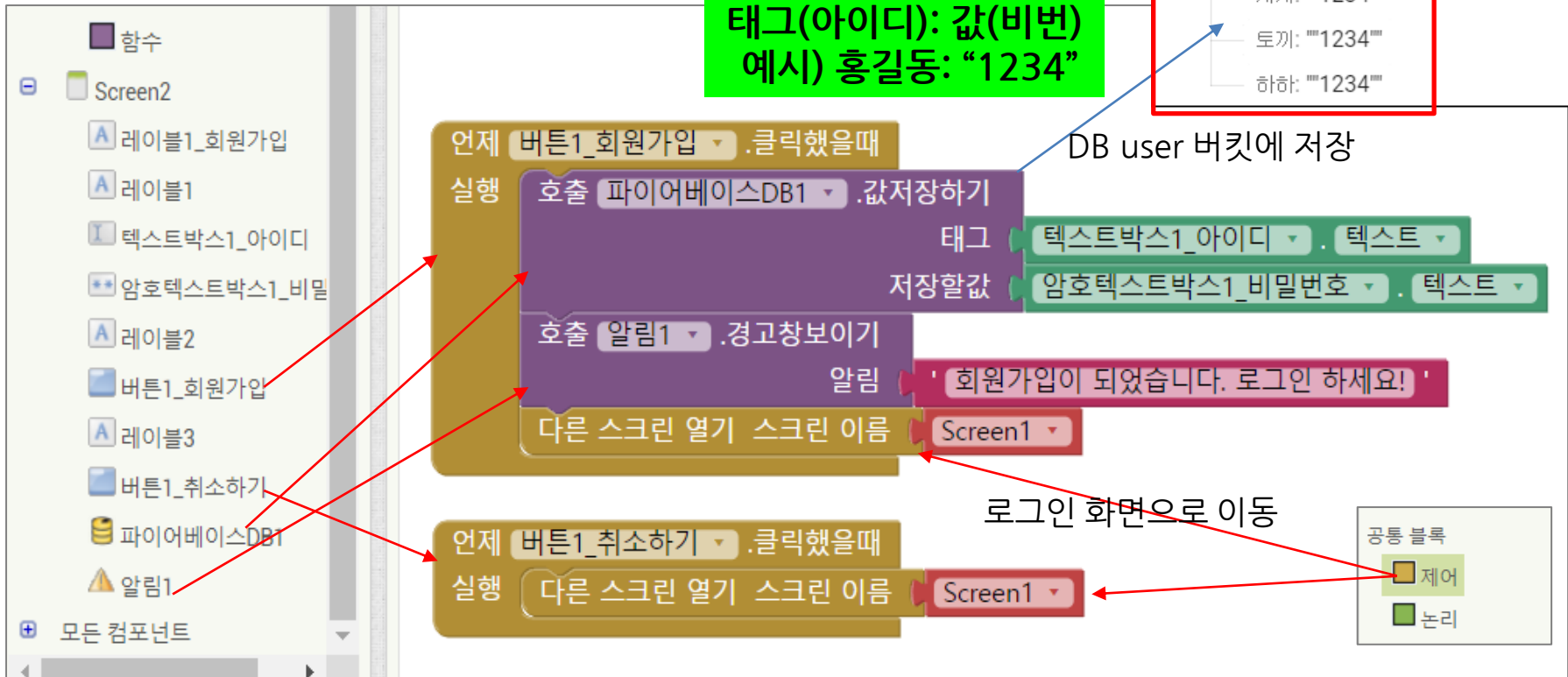
■ 그룹채팅 앱 제작하기 - Screen2(회원가입) 코딩

- 아이디와 비밀번호 등록하여 회원가입하기
 - DB의 **user** 버킷에 태그와 값으로 저장



태그(아이디): 값(비번)
예시) 홍길동: "1234"

DB user 버킷에 저장



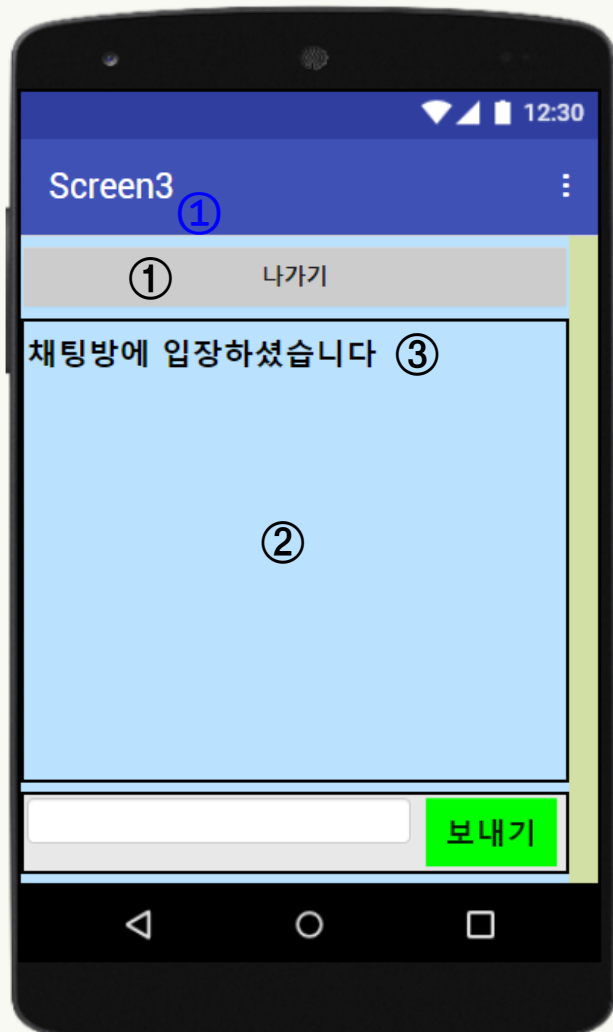
4. [그룹 채팅 앱] 제작 - Screen3 디자인

■ 그룹채팅 앱 제작하기 - Screen3(그룹채팅) 디자인



	컴포넌트	이름	속성
	Screen3	Screen3	수평정렬- 왼쪽, 수직정렬 - 위, 배경색- 자유
①	버튼	버튼1_나가기	글꼴굵게, 글꼴크기-15, 너비-부모요소, 모양-직사각형, 텍스트-나가기
②	스크롤가능 수직배치	스크롤가능수직배치1	수평정렬- 왼쪽, 수직정렬-위 높이 - 부모요소, 너비-부모요소
③	레이블	레이블_채팅내용	글꼴굵게, 글꼴크기-20 텍스트 - 채팅방에 입장하셨습니다
④	수평배치	수평배치1	수평정렬-왼쪽, 너비- 부모요소
	텍스트박스	텍스트박스1_메시지 입력	글꼴굵게, 글꼴크기-20, 너비 - 70% 힌트- 메시지 입력
	버튼	버튼1_전송하기	배경색- 자유, 글꼴굵게, 글꼴크기-20 텍스트-보내기
⑤	시계	시계1	타이머활성화여부 - 체크해제 타이머간격 - 1000 (1초)
⑥	파이어베이스DB	파이어베이스DB1	파이어베이스URL - https://networkdesignproject-f90ba-default-rtdb.firebaseio.com/ 프로젝트버킷 - chat

※ [스크롤 가능 수직배치] 레이아웃은 수직으로 화면을 벗어날 만큼의 컴포넌트들이 배치 되었을 때 상하로 스크롤이 가능하게 해 주는 레이아웃



보이지 않는 컴포넌트

⑤



시계1 파이어베이스DB1

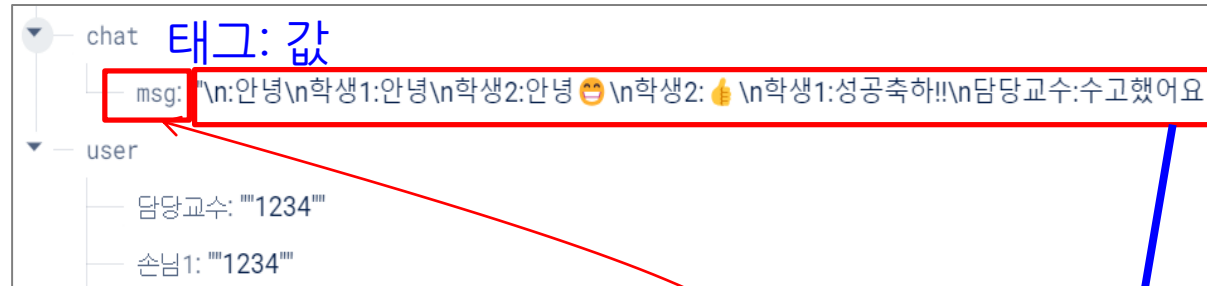
⑥

4. [그룹 채팅 앱] 제작 - Screen3 코딩

■ 그룹채팅 앱 제작하기 - Screen3(그룹채팅) 코딩

- 주기적으로 DB에 저장된 채팅 메시지 가져와서 앱 화면에 출력

※ [시계(타이머)] 컴포넌트의 역할:
1초마다 DB에 저장된 채팅메시지
가져와서 앱에 출력하기 위해
시계(타이머) 사용

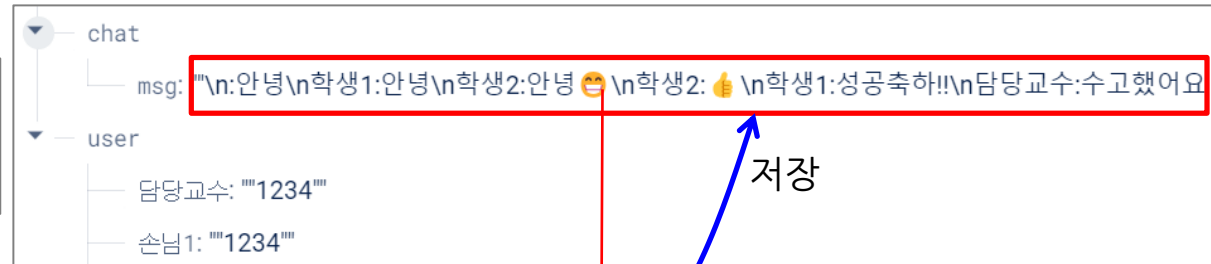


4. [그룹 채팅 앱] 제작 - Screen3 코딩

■ 그룹채팅 앱 제작하기 - Screen3(그룹채팅) 코딩

○ 채팅 메시지 DB에 저장하기

※ 주의) 이 부분이 중요합니다. 잘못 코딩 시 공용 DB에 저장된 전체 채팅 메시지가 훼손될 수 있습니다



DB에 저장된 이전 메시지에다 신규메시지 추가하여 저장하는 방식



5. [그룹 채팅 앱] 제작 - 실행 및 테스트

■ 그룹채팅 앱 제작하기 - 실행 및 테스트



14주차 과제 안내

■ 과제명: 과목 프로젝트인 [그룹채팅 앱] 제작하기

- 제출방법: [그룹채팅 앱] 프로젝트 파일을 제출해 주세요
- 첨부파일: GroupChat_학번.aia
- 제출일자: 6/8(월) 수업시간 내
- 필수 기능이 정상 동작하는지 확인 후 제출해 주세요.

한 학기 네트워크 실습으로 수고 많았습니다!

- 네트워크의 이해와 네트워크 프로그래밍은 개발자의 주요역량
 - 네트워크 프로그램은 서버와 클라이언트 간의 통신 프로그램
 - 웹 서비스, 게임, 메시지 애플리케이션 등 다양한 시스템을 개발하는 데 필수적인 기술
 - 서버와 클라이언트 간의 통신을 이해하고 구현하는 능력은 실무에서 매우 중요
 - 향후 여러분이 현업에서 프로그램 개발에 또는 프로그램 유지보수에 네트워크설계 수업이 도움이 되기를 바랍니다

한 학기 수고 많았고 열심히 참여해주어 고맙습니다!

네트워크 설계 기말고사 안내

□ 시험일자: 6/15(월) , 수업시간

□ 시험범위

- 수업시간에 다룬 내용만 출제되며, 강의자료 위주로 공부하세요.
- 강의자료는 온라인학습사이트(ctl.gtec.ac.kr)에서 주차별 교안을 다운받으면 됩니다.

구분	주제	주요내용	비중
9주차	웹과 HTTP 개요 (기반지식)	- HTTP 개요 - HTTP 요청메시지와 응답메시지 구조 이해 - 요청 메소드, 상태코드 이해	대략 50%
11주차	파이썬 http.server 모듈을 이용한 웹 서버 구현	- http.server 모듈을 이용한 서버 프로그래밍 이해	대략 20%
12주차	파이썬 requests 모듈 사용하기	- 파이썬 requests 모듈 사용법 이해 - get() 방식과 post() 방식 차이이해	
13주차	파이썬 Flask웹 프레임워크	- Flask 기본 사용법과 변수 사용법 이해	대략 15%
	그룹채팅 앱 제작	- 그룹채팅 앱 디자인 과 코드 이해(동작방식)	대략 15%

※ 객관식(단답형포함)과 주관식

※ 9주차 HTTP의 기반지식을 잘 이해하시기 바랍니다 (객관식 출제)

※ 코드는 지문으로 제시하고 관련 코드의 동작방법, 관련된 용어 문제
(강의자료 뒷부분 요약정리 파트에 관련 코드는 제시되어 있습니다)

※ 용어설명: HTTP 관련 용어 설명

9~14주차까지 실습한 내용 중 핵심 사항 위주로 정리한 자료입니다.

기말고사 시험 대비하여

주차별 핵심 내용은 꼭 이해하시기 바랍니다.

1. HTTP (HyperText Transfer Protocol) 개요

■ 웹(Web) 개요

- 인터넷을 이용해 할 수 있는 기능 중의 하나 (인터넷 >> 웹)
- 멀리 떨어져 있는 컴퓨터간 문서를 주고 받기 위해 탄생 (1990년)
- 웹의 4가지 구성요소
 - ① 웹 브라우저와 웹 서버 - 웹 페이지를 주고받는 소프트웨어
 - ② HTML - 웹 페이지를 만드는 언어
 - ③ URL - 원하는 웹 페이지가 어디에 있는지 알려주는 주소체계
 - ④ HTTP - 웹 브라우저와 웹 서버가 통신할 때 사용하는 통신규칙

1. HTTP (HyperText Transfer Protocol) 개요

■ URL Uniform Resource Locator

- 웹 주소, 네트워크 상에서 리소스가 어디 있는지 알려주기 위한 주소체계
- URL 형식: 프로토콜 // IP 주소 또는 도메인 주소 : 포트번호 / 자원 위치

* 아래는 네이버 지도에서 '경기과학기술대학교' 검색시의 URL이다

[https:// map.naver.com \(:443\) /p/search/경기과학기술대학교 ?c=15.00,0,0,2,dh](https://map.naver.com(:443)/p/search/경기과학기술대학교?c=15.00,0,0,2,dh)

Protocol	host	port	path	Query string
----------	------	------	------	--------------

- ① Protocol(프로토콜): 어떤 프로토콜을 사용하는지를 알려주는 역할 (http 또는 https)
- ② Host(호스트): 해당 웹사이트의 도메인 주소(네임 서버)
- ③ Port(포트) : URL엔 생략되어 있음 (http는 80, https는 443을 기본포트)
- ④ Path(경로): 해당 웹사이트의 더 자세한 경로를 나타내는 역할
- ⑤ Query(쿼리스트링) : URL에서 "?" 뒤에 조건에 맞는 입력 파라미터를 전달하기 위해 사용.
입력한 값에 따라 다른 결과를 보여줘야 할 때 쿼리스트링을 사용

1. HTTP (HyperText Transfer Protocol) 개요

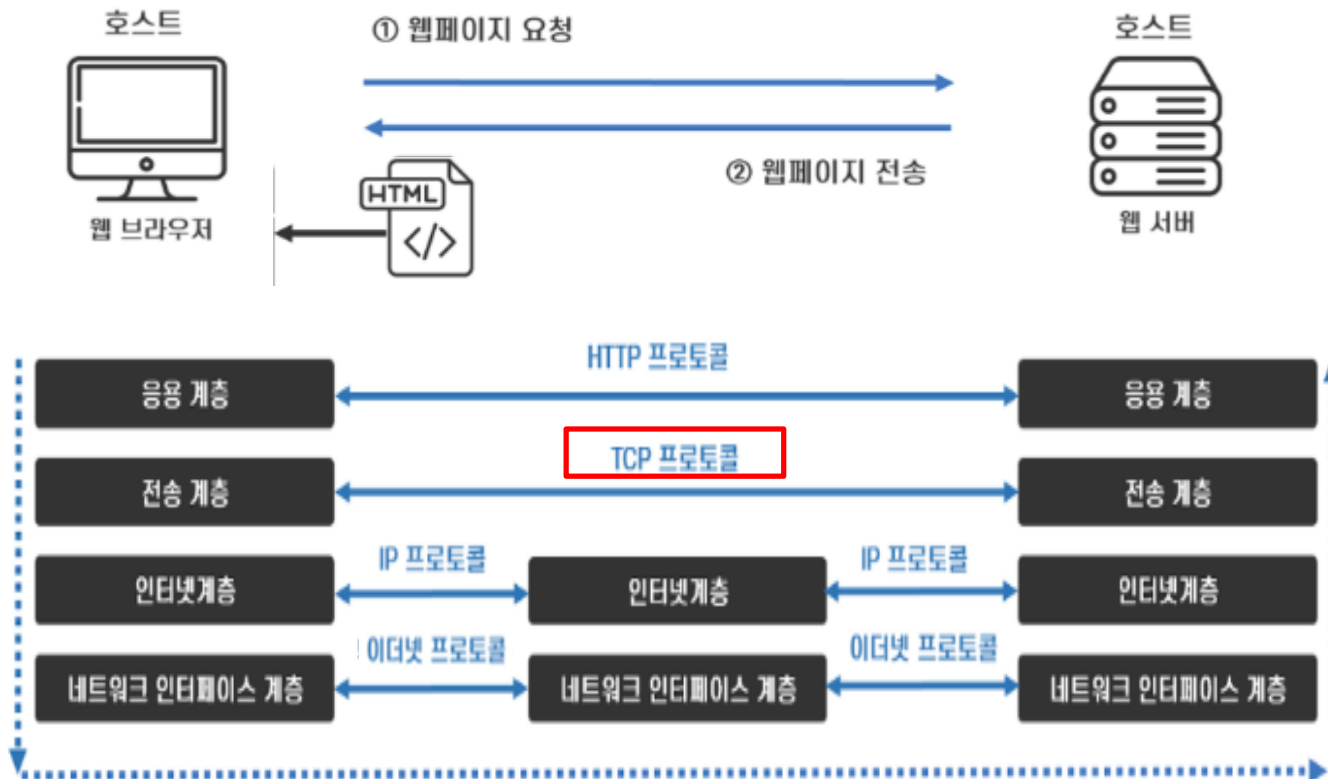
■ HTTP 개요

- 서버와 클라이언트간 HTML 문서를 주고 받기 위한 통신 규칙으로

웹 브라우저의 요청(Request) 메시지와 웹 서버의 응답(Response) 메시지의 구조를 규정

- HTTP (HyperText Transfer Protocol)과

HTTPS (HyperText Transfer Protocol over Secure Socket Layer): 보안이 강화된 HTTP

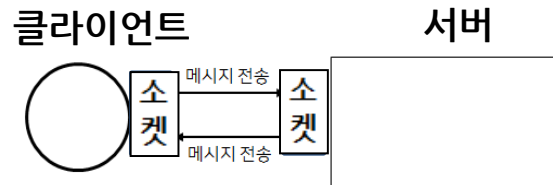


1. HTTP (HyperText Transfer Protocol) 개요

■ HTTP 개요

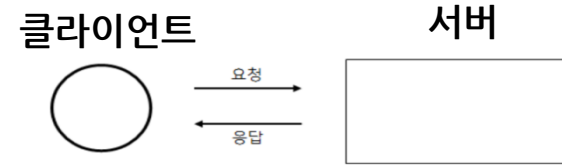
○ HTTP 통신과 소켓 통신의 차이

소켓 통신



- 서버와 클라이언트 **양방향 통신**
:클라이언트와 서버 양쪽에서 서로에게 데이터를 전달하는 방식의 양방향 통신
- 서버나 클라이언트에서 강제로 접속을 해제할 때 까지는 계속 **연결을 유지하는 방식**
: 클라이언트 수가 늘어날 수록 서버의 부하 증가 → 접속할 수 있는 클라이언트 수 제한

HTTP 통신



- 클라이언트 요청이 있을 때 서버가 응답하는 방식. **단방향 통신**
:클라이언트에서 서버로 요청을 보내고 서버는 응답하는 방식
- JSON, Image, HTML 파일 등 다양한 파일을 전송 받을 수 있음
- 응답을 받은 후 연결이 끊어지는 것이 기본 동작(HTTP 1.0)
- 성능 상으로 필요하다면 Keep Alive 옵션을 주어 일정 시간 동안 연결을 유지하는 것이 가능 (HTTP 1.1 이상)

“HTTP 통신은 소켓통신의 일종”

1. HTTP (HyperText Transfer Protocol) 개요

■ HTTP 개요

○ HTTP 특징

① 서버와 클라이언트 구조

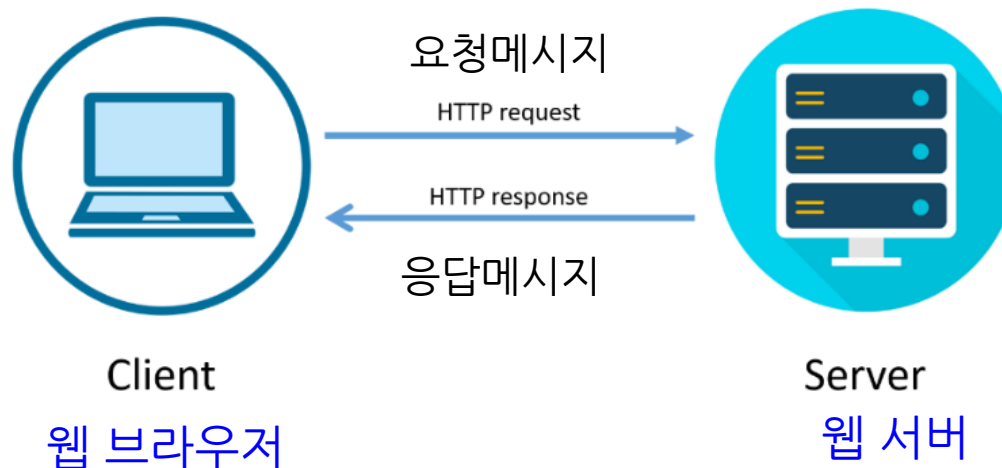
② 무상태 프로토콜(Stateless)

- 서버는 클라이언트 요청정보에 대해서만 응답하며, 이전 요청과는 무관

③ 비연결성 (Connectionless) 프로토콜

- HTTP는 기본적으로 연결을 유지하지 않는 모델
단, HTTP 1.1부터 HTTP 지속 연결 가능 (Persistent Connections)

`https://comic.naver.com/index`



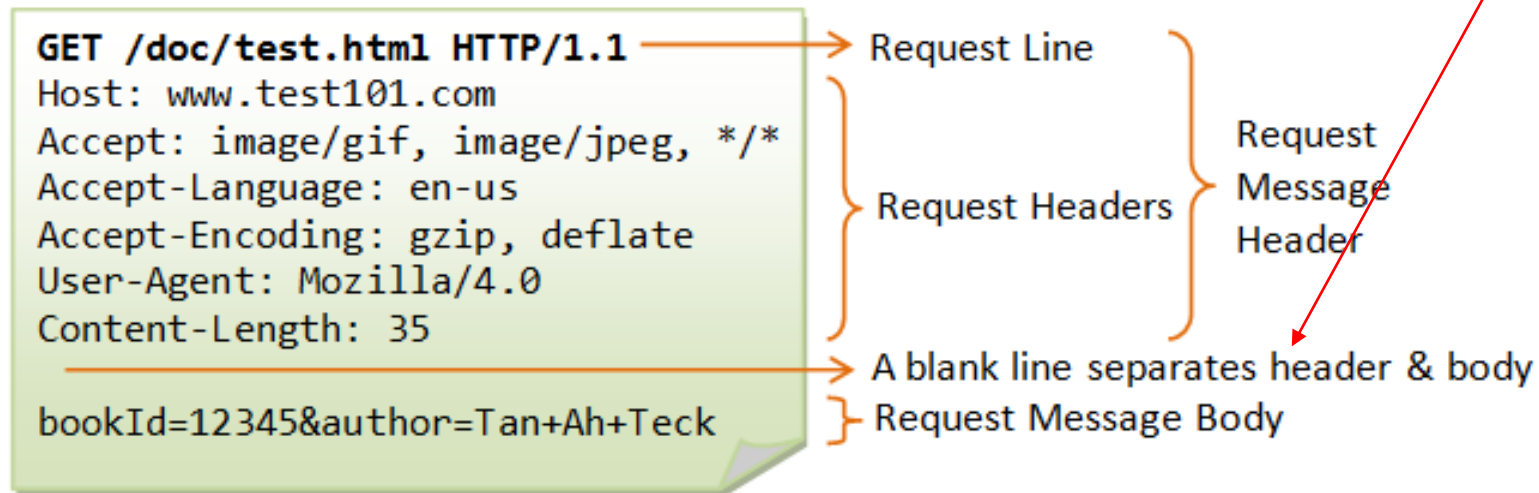
1. HTTP (HyperText Transfer Protocol) 개요

■ HTTP 요청 메시지 (Request Message) 구조

○ HTTP Request Message의 구성

- Request Line
- Headers
- blank line (공백)
- Body

헤더와 바디를 구분하기 위해
헤더와 바디 사이에
blank line(공백)이 있음
→ **CRLF(\r\n)** : 공백라인



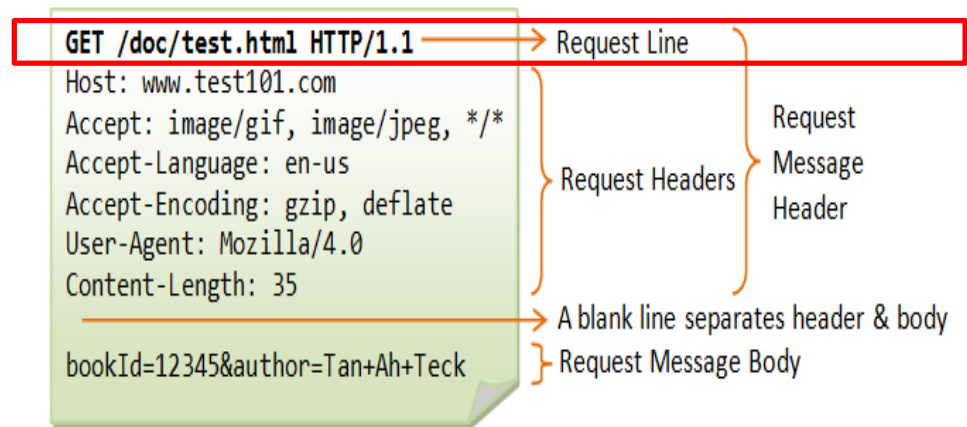
그림출처: gnaseel.tistory.com

1. HTTP (HyperText Transfer Protocol) 개요

■ HTTP Request Message 구조

Request line

- HTTP Request Message의 시작 라인
- 3가지 부분으로 구성
 - ① HTTP method
 - ② URL
 - ③ HTTP version



GET /doc/test.html HTTP/1.1
[HTTP Method] [URL] [HTTP version]

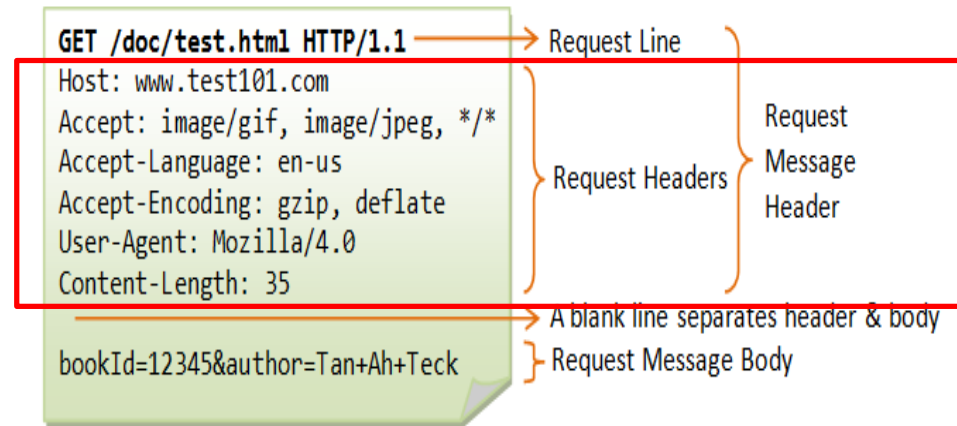
- ① HTTP Method : 클라이언트가 웹 서버에게 요청하는 목적을 알리는 수단
종류) GET 방식, POST 방식등
 - GET은 서버로 정보를 요청하는 목적 (“서버야 ~~ 좀 줘!”)
 - POST는 서버로 정보를 전달하는 목적 (“서버야 ~~ 줄 테니 처리해 줘”)
- ② URL : 요청 자원 명시
- ③ HTTP version: HTTP version을 명시(HTTP 1.0, 1.1, 2.0 등)

1. HTTP (HyperText Transfer Protocol) 개요

■ HTTP Request Message 구조

Headers

- 해당 request에 대한 추가 정보를 담고 있는 부분



HTTP 요청 메시지에서 사용하는 헤더 항목

* *Header*는 *Request Line*에서 표현되지 않은 더 구체적인 요구(추가정보)를 작성하는 공간

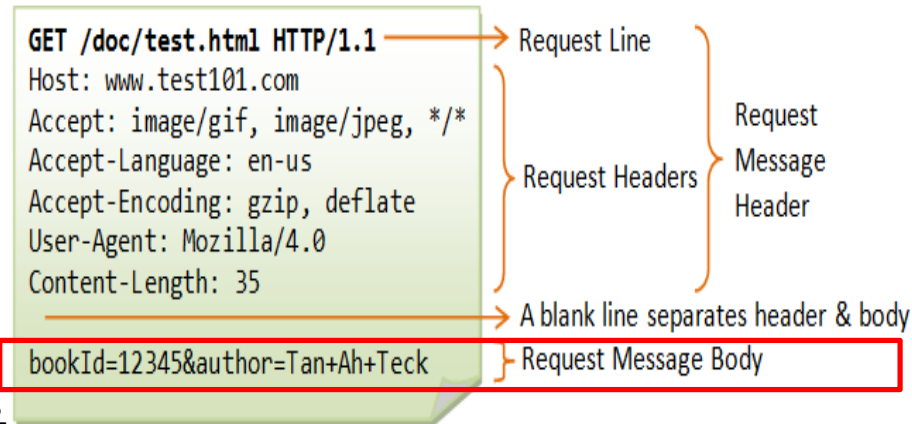
- **Host** : 서버의 도메인 주소
- **Connection** : Keep-alive가 디폴트 (HTTP/2에선 사용하지 않음)
- **User-Agent** : 사용자의 웹 브라우저 종류&버전 정보
- **Accept** : 브라우저가 처리할 수 있는 데이터의 형태
- **Accept-Language** : 사용하는 언어
- **Accept-Encoding** : 브라우저가 처리할 수 있는 콘텐츠 압축 방식

1. HTTP (HyperText Transfer Protocol) 개요

■ HTTP Request Message 구조

Body

- HTTP Request가 전송하는 데이터를 담고 있는 부분
- 전송하는 데이터가 없다면 body 부분은 비어있음 즉, GET 요청 시에는 Body는 비어있고 POST 요청일 경우 Body에 정보가 포함되어 있음



1. HTTP (HyperText Transfer Protocol) 개요

■ HTTP 응답메시지(Response Message) 구조

○ HTTP Response Message의 구성

- Status Line
- Headers
- Blank line(공백)
- Body

헤더와 바디를 구분하기 위해
헤더와 바디 사이에
blank line(공백)이 있음
→ **CRLF(\r\n)** : 공백라인



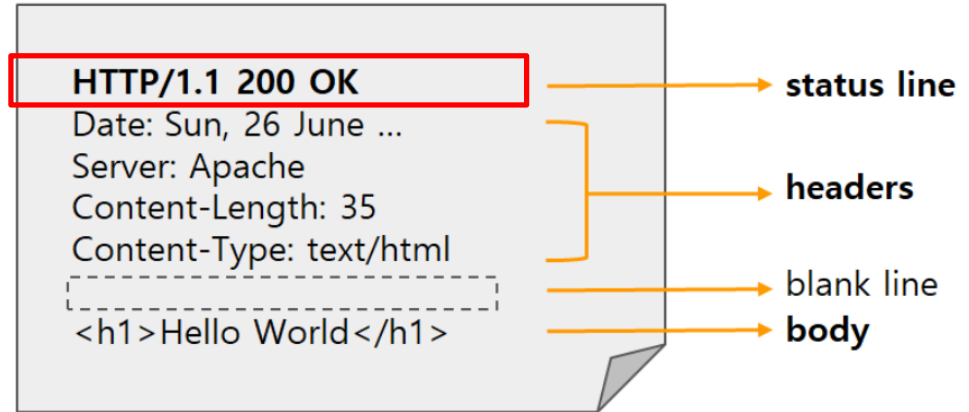
1. HTTP (HyperText Transfer Protocol) 개요

■ HTTP Response Message 구조

Status line

- HTTP Response의 상태를 나타내는 부분
- 3가지 부분으로 구성

- ① HTTP version
- ② Status Code
- ③ Status Text



HTTP/1.1 200 OK
[HTTP version] [Status Code] [Status Text]

HTTP Status Codes		
Level 200	Level 400	Level 500
200: OK	400: Bad Request	500: Internal Server Error
201: Created	401: Unauthorized	501: Not Implemented
202: Accepted	403: Forbidden	502: Bad Gateway
203: Non-Authoritative Information	404: Not Found	503: Service Unavailable
204: No content	409: Conflict	504: Gateway Timeout
		599: Network Timeout

3xx Redirection	
301	Moved Permanently
302	Found
303	See Other
304	Not Modified

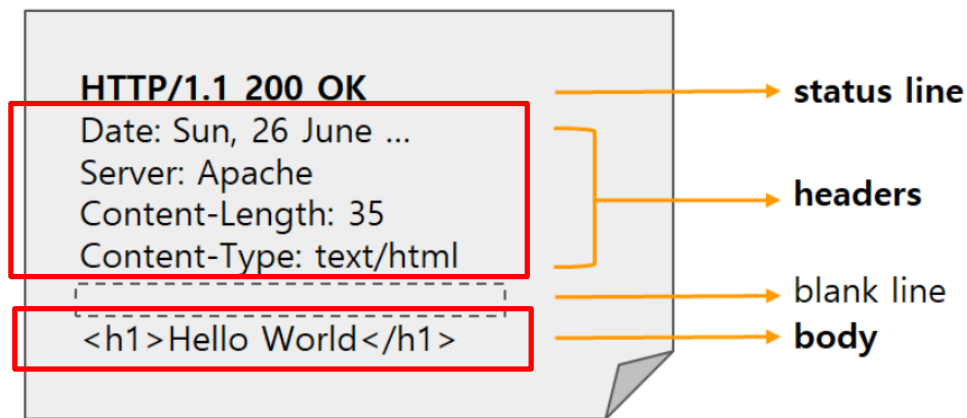
캐시관련: 바뀐 게 없으니
캐시 데이터 그냥 사용해!

1. HTTP (HyperText Transfer Protocol) 개요

■ HTTP Response Message 구조

Headers

- Response의 headers와 거의 유사



- 네트워크상에서 데이터를 보낼 때 데이터의 형식을 나타내는 표준방식인 **MIME** 타입 사용

*MIME Multipurpose Internet Mail Extensions : 미디어 타입 (주타입/서브타입)

서버는 모든 HTTP 객체에 **MIME** 타입을 붙임

- **Content-type**: 보내는 자원의 형식을 명시하기 위해
헤더에 실리는 정보 (**MIME** 타입)
예) Content-Type: text/html : text 중 html 파일
- **Content-Length** : body의 길이(바이트 단위)

Body

- 보통은 클라이언트가 요청한 HTML 문서가 들어감

유형	MIME Type	주요 용도
문서	text/html, text/plain	웹 페이지 및 일반 텍스트
스타일	text/css	디자인 (CSS)
스크립트	text/javascript	동적 기능 (JS)
데이터	application/json	API 데이터 교환
이미지	image/png	투명 지원 이미지
비디오	video/mp4	표준 영상 파일
기타	application/pdf	문서 배포

2. Python http.server 모듈을 이용한 웹 서버 구현

■ Python http.server 모듈

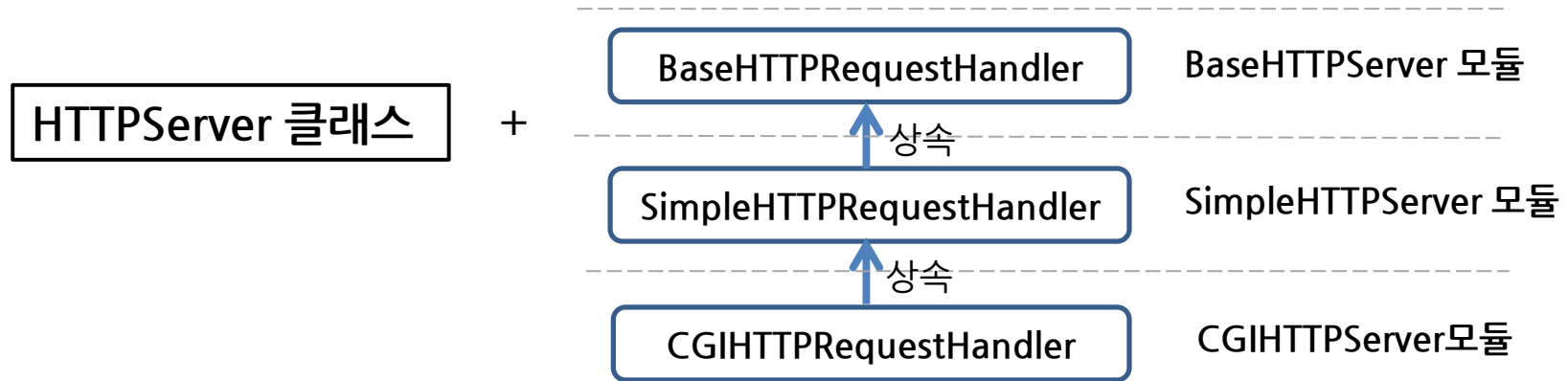
- http.server 모듈은 파이썬으로 HTTP 서버를 구현하기 위한 기본 모듈
별도의 외부 라이브러리 없이 파이썬 만으로 간단한 웹 서버를 구축할 수 있게 해주는 내장 모듈
 - 핵심 클래스 2개
 - 1) HTTPServer 클래스: 웹 서버 객체를 만들기 위한 클래스
 - 객체 생성시 서버 주소정보(IP와 PORT)와 핸들러 클래스 반드시 필요
 - 2) 핸들러 클래스 : 어떤 요청이 들어올 때 어떻게 응답할지 정의
 - BaseHTTPRequestHandler 또는 SimpleHTTPRequestHandler 클래스
- HTTPServer 클래스와 핸들러 클래스를 사용하여 직접 서버 동작을 사용자 정의할 수 있음

참조2. Python http.server 모듈을 이용한 웹 서버구현

■ http.server 모듈

- 파이썬은 HTTP 서버를 구현하기 위한 기본 모듈

핸들러 클래스



- http.server가 제공하는 핸들러의 종류(어느 정도 구현이 되어 있는가의 차이)

종류	핸들러 클래스	특징
가장 기본 기본적인 프레임만 제공, 사용자가 로직 직접 추가	<code>BaseHTTPRequestHandler</code>	- 어떤 명령 메소드도 구현되지 않은 기본 서버 *메소드 오버라이딩(Method Overriding) 사용 :부모 클래스의 메소드를 상속받아 자식 클래스에서 재정의하는 방식
정적 파일 서버 GET은 구현되어 있음	<code>SimpleHTTPRequestHandler</code>	- <code>BaseHTTPServer</code> 의 서브클래스 - GET, HEAD method 구현된 서버
스크립트 실행용 GET, POST 구현	<code>CGIHTTPRequestHandler</code>	- <code>SimpleHTTPServer</code> 의 서브클래스 - POST method구현

참조2. HTTP 서버 모듈을 이용한 HTTP 서버 프로그래밍

■ HTTP 요청 메소드

메소드	특징
GET	- 리소스를 얻어 오기 위한 요청 - 바디 사용 없이 URL에 쿼리 스트링으로 데이터 전달
POST	- 새로운 리소스를 생성하기 위한 요청 - 바디를 사용해 데이터 전달
PUT	- 리소스 수정 요청(원래 자원에서 새로운 정보 추가, 삭제하기 위한 요청) - 바디를 사용해 데이터 전달
DELETE	- 리소스를 삭제하기 위한 요청 - 바디 사용 없이 쿼리 스트링으로 데이터 전달

참조2. HTTP 서버 모듈을 이용한 HTTP 서버 프로그래밍

- 코드1) http.server 모듈을 이용한 간단한 웹 서버 구현하기 (BaseHTTPRequestHandler 사용)

```
1 # http.server 모듈을 이용한 간단한 웹 서버 구현
2 from http.server import HTTPServer, BaseHTTPRequestHandler
3
4 # 웹 페이지 작성(index.html)
5 html_index = '''
6     <html>
7         <body>
8             <h1>This is simple my HTTP Server using http.server</h1>
9         </body>
10    </html>
11    '''
12
13 # base 핸들러 상속받아 자체 핸들러 구현: 클라이언트 요청에 대한 처리 담당
14 class myHandler(BaseHTTPRequestHandler):
15     # 서버로 들어오는 모든 URL의 GET 요청에 대해 동일한 웹 페이지 전송
16     def do_GET(self): # 빈 로직을 상속받아 메소드 오버라이딩으로 자체 구현
17         print("GET 요청들어옴")
18         self.send_response(200) # status line(HTTP/1.1 200 OK)
19         self.send_header("Content-type", "text/html; charset=utf-8") # 응답바디의 타입지정
20         self.end_headers() # 더 이상 헤더없음. blank line(\r\n) 추가
21         # 클라이언트로 전송할 응답 바디 전송
22         self.wfile.write(html_index.encode('utf-8'))
23
24 if __name__ == "__main__":
25     address = ('localhost', 8080)
26     # HTTP 서버 생성: HTTPServer(주소, 핸들러)
27     server = HTTPServer(address, myHandler)
28     print(f"웹 서버가 {address}로 서비스 되고 있습니다")
29     server.serve_forever() # 서버시작(looping)
```

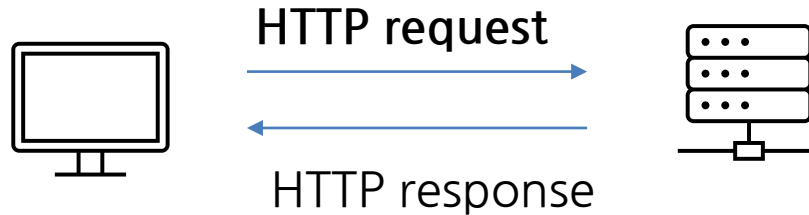
This is simple my HTTP Server using http.server

3. 파이썬 requests 모듈 사용하기

- Python requests 모듈 사용하기

- requests 모듈은 Python용 HTTP 라이브러리

- HTTP 요청을 보내고 응답을 받는 데 사용되는 라이브러리



- 각 요청방식(Method)에 맞는 함수 제공

GET 방식 – requests.get()

POST 방식 – requests.post()

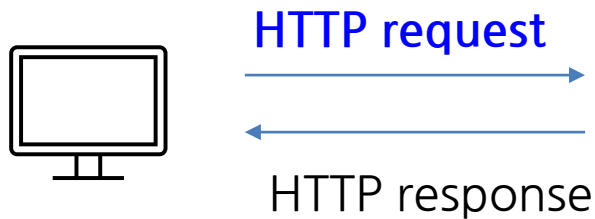
PUT 방식 – requests.put()

DELETE 방식 – requests.delete()

참조3. 파이썬 requests 모듈 사용하기

■ Python requests 모듈 사용하기

- <https://httpbin.org> 를 이용해서 requests 모듈 사용법을 익혀봅시다
 - * *httpbin.org* 는 *http* 메서드 테스트 사이트 (보낸 요청을 그대로 돌려줌)



httpbin.org 서버를 이용해
GET 방식과 POST 방식 요청 시
입력 데이터가 어떻게 서버로 전달되는지
이해해 봅시다

httpbin.org 0.9.2
[Base URL: httpbin.org/]
A simple HTTP Request & Response Service.
Run locally: `$ docker run -p 80:80 kennethreitz/httpbin`
[the developer - Website](#)
[Send email to the developer](#)

Schemes

HTTPS ▼

HTTP Methods Testing different HTTP verbs

DELETE	/delete	The request's DELETE parameters.
GET	/get	The request's query parameters.
PATCH	/patch	The request's PATCH parameters.
POST	/post	The request's POST parameters.
PUT	/put	The request's PUT parameters.

참조3. 파이썬 requests 모듈 사용하기

■ Python requests 모듈 사용하기 - GET 방식

1) GET 방식으로 파라미터 데이터 보내기 : `requests.get(url, params)`

- 파라미터는 URL의 쿼리 스트링으로 전송 (`url + ?query 스트링`)

```
# GET 요청시 파라미터 추가하기
# requests.get(url, params)

import requests, json

url = 'https://httpbin.org/get'
# 파라미터 정의(key:value)
params = {"name": "Hello", "data": "World"} #서버로 보낼 파라미터 데이터 정의

# GET 요청시 파라미터 추가- requests.get(url, params)
response = requests.get(url, params)
dic_data = response.json() # JSON -> Dic
print("GET요청 URL:", dic_data['url'])
```

GET요청 URL: <https://httpbin.org/get?name=Hello&data=World>

URL의 query 스트링에 파라미터 데이터 실어서 전달 (? key1=value1&key2=value2...)

■ Python requests 모듈 사용하기 - POST 방식

2) POST 방식으로 파라미터 데이터 보내기 : requests.post(url, data)

- 파라미터는 요청메시지의 body에 싸서 폼 데이터 형식으로 전송

```
# POST 요청시 파라미터 추가하기
```

```
# requests.post(url, data)
```

```
import requests, json
```

```
url = 'https://httpbin.org/post'
```

```
# 요청메시지의 바디 정의(일반 폼 데이터 형식)
```

```
body_data = {"name":"Hello", "data":"World"} #서버로 보낼 파라미터 데이터 정의
```

```
# POST 요청으로 바디를 보내고 응답받기
```

```
# requests.post(url, body_data)
```

```
response = requests.post(url, data=body_data)
```

```
dic_data = response.json() # JSON -> Dic
```

```
print("POST 요청 URL:", dic_data['url'])
```

```
print("POST 요청 바디:", dic_data['form'])
```

```
POST 요청 URL: https://httpbin.org/post
```

```
POST 요청 바디: {'data': 'World', 'name': 'Hello'}
```

URL에 파라미터는 없고
요청메시지 body에 실려 서버로 전송

■ Flask 기초

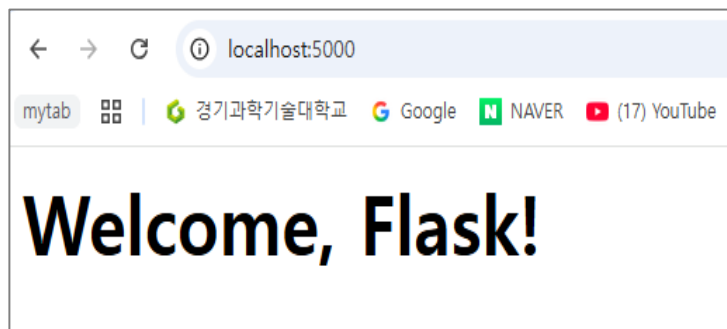
- Flask는 파이썬 웹 프레임워크
 - Flask는 파이썬을 기반으로 한 **마이크로 웹 프레임워크**
 - django는 파이썬을 기반으로 한 풀 웹 프레임워크
- **웹 프레임워크**(Framework: 틀, 도구상자)란?
 - 웹 사이트나 웹 애플리케이션을 개발할 때 기본적인 구조와 기능들을 제공하여 개발자가 쉽고 빠르게 웹사이트를 개발할 수 있도록 돕는 도구

1. Python Flask Web Framework

■ 실습1) Flask 기초 : 기본 코딩(1)

- Flask 애플리케이션을 생성하기 위한 기본 코드 (루트경로: http://localhost:5000)
: flask 객체 생성 → 라우팅 설정 → 서버 실행

```
1 # Flask 웹 프레임워크 기본 사용법
2 from flask import Flask
3
4 # 1. Flask 웹 서버 객체 생성
5 # 현재 파일을 기준으로 Flask 웹 서버 객체를 만들어서 app 변수에 저장
6 app = Flask(__name__)
7
8 # 2. 라우팅 설정: 어떤 주소(경로)로 들어왔을 때, 어떤 함수를 실행할지 연결해 주는 것
9 # 사용자가 홈페이지 접속('/')했을 때 실행할 함수 지정
10 @app.route('/')
11 def index():
12     return "<h1>Welcome, Flask!</h1>"
13
14 # 3. 서버 실행 (로컬 호스트에서 실행)
15 # flask 기본포트는 5000번
16 if __name__ == '__main__':
17     app.run(host='localhost', port=5000)
```

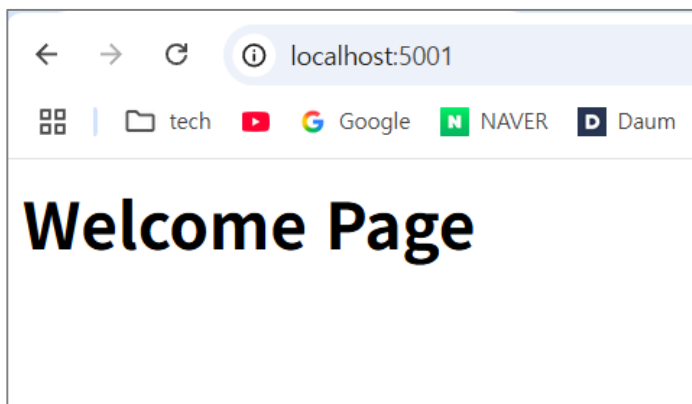


1. Python Flask Web Framework 기초

■ Flask 기초 : 라우팅

- Flask 라우팅: 사용자가 요청한 URL 경로에 따라 수행할 함수를 연결해 주는 기능
- “이 주소로 요청이 들어온 경우 밑에 있는 함수를 통해 이 화면을 보여주자!”

```
@app.route('/')
```

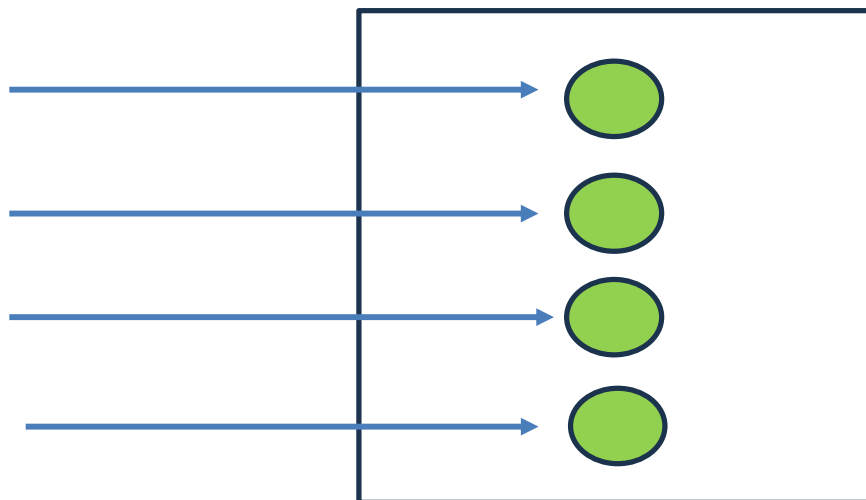


http://localhost:5000

http://localhost:5000/read

http://localhost:5000/create

http://localhost:5000/update

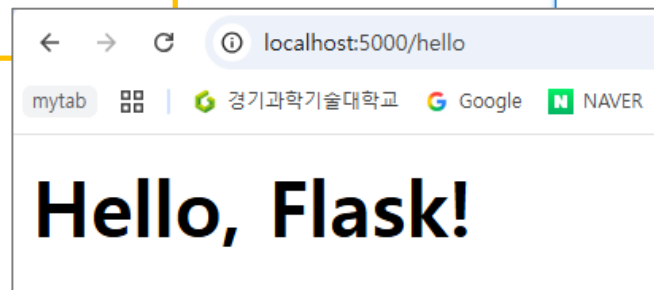


1. Python Flask Web Framework

■ 실습1) Flask 기초 : 기본 코딩(2)

- 라우팅 추가하기 (경로: <http://localhost:5000/hello>)

```
1 # Flask 웹 프레임워크 기본 사용법
2 from flask import Flask
3
4 # 1. Flask 웹 서버 객체 생성
5 # 현재 파일을 기준으로 Flask 웹 서버 객체를 만들어서 app 변수에 저장
6 app = Flask(__name__)
7
8 # 2. 라우팅 설정: 어떤 주소(경로)로 들어왔을 때, 어떤 함수를 실행할지 연결해 주는 것
9 # 사용자가 홈페이지 접속('/')했을 때 실행할 함수 지정
10 @app.route('/')
11 def index():
12     return "<h1>Welcome, Flask!</h1>"
13
14 ## 사용자가 경로 ('/hello')에 접속했을 때 실행할 함수 지정
15 @app.route('/hello/')
16 def hello():
17     return "<h1>Hello, Flask!</h1>"
18
19 # 3. 서버 실행 (로컬 호스트에서 실행)
20 # flask 기본포트는 5000번
21 if __name__ == '__main__':
22     app.run(host='localhost', port=5000)
```



1. Python Flask Web Framework

■ 실습1) Flask 기초 : 기본 코딩(3)

- 라우팅 경로에 <변수명> 형식으로 처리하기 (경로: <http://localhost:5000/hello/name>)

예) <http://localhost:5000/hello/name>

- hello 뒤에 오는 이름을 변수로 받아서 동적으로 웹 페이지를 제공해 봅시다.

기존 코드에 추가해 봅시다.

- 라우팅에서 <변수명>로 처리하고, 함수에서 인자로 받아오기

```
19 # 4. 라우팅 경로에 <변수명> 사용하기
20 @app.route('/hello/<name>/') # URL 경로에 <변수명> 추가
21 def sayHello(name):          # 변수명을 함수의 인자로 전달
22     # 동적으로 웹 페이지 변경
23     return f"<h1>Hello, {name}님<br><br>Flask의 세계에 오신것을 환영합니다!</h1>"
```

← → ↺ ⓘ localhost:5000/hello/홍길동/

Hello, 홍길동님

Flask의 세계에 오신것을 환영합니다!

2. Python Flask GET, POST 메소드 처리

■ 실습2) Flask GET, POST 메소드 처리

- GET 요청 : 모든 파라미터를 URL의 쿼리스트링으로 보내 요청하는 방식
“http://localhost:5000/get/?user_name=홍길동”
- Flask는 기본적으로 GET방식으로 동작
- 쿼리스트링에서 key의 value를 가져오려면, request.args.get(key)

```
33 # 라우팅 설정
34 # 루트('/')로 접속시 홈 화면 전송하기
35 @app.route('/')
36 def index():
37     return html_form
38
39 # 경로('/get')으로 GET방식 요청시 처리
40 @app.route('/get/', methods=['GET'])
41 def hello_get():
42     # URL 쿼리스트링에서(?user_name=value) value 데이터(이름) 가져오기
43     name = request.args.get('user_name')
44     return f'''
45         <h3>[GET 요청 처리화면]</h3>
46         <h1>안녕하세요, {name}님!</h1>
47         <p>주소창(URL)에 입력한 이름이 그대로 보입니다</p>
48         <a href='/'>돌아가기 </a>
49     '''
50
```

localhost:5000/get/?user_name=홍길동

GET 방식으로 파라미터(이름) 보내기

이름:

GET 전송

[GET 요청 처리 화면]

안녕하세요, 홍길동님!

주소창(URL)에 입력한 이름이 그대로 보입니다

[돌아가기](#)

2. Python Flask GET, POST 메소드 처리

■ 실습2) Flask GET, POST 메소드 처리

- POST 요청: 전달하려는 정보가 HTTP 요청 body에 포함되어 전달
 - 전달하려는 정보를 body(Form Data)에 실어서 post 방식으로 전달하는 방법

{'user_name':'홍길동'}이 응답 body에 실려서 전달

- Flask로 POST 처리하려면 라우팅에서 POST를 추가해야 함. (추가하지 않으면 404 Not found!)
- 요청메시지 body에서 key의 value를 가져오려면, `request.form.get(key)`

```
51 # 경로('/post')으로 POST방식 요청시 처리
52 # flask는 기본적으로 GET방식으로 처리. POST방식 처리하려면
53 @app.route('/post/', methods=['POST'])
54 def hello_post():          POST 추가
55     # 요청메시지 body(form 데이터)에서 value 데이터(이름) 가져오기
56     name = request.form.get('user_name')
57     return f'''
58         <h3>[GET 요청 처리화면]</h3>
59         <h1>안녕하세요, {name}님!</h1>
60         <p>주소창(URL)에 입력한 이름이 그대로 보입니다</p>
61         <a href='/'> 돌아가기 </a>
62     '''
63 # flask 웹 서버 실행
64 if __name__ == "__main__":
65     app.run(host="localhost", port=5000)
```

POST 방식으로 파라미터(이름) 보내기

아이디:

POST 전송

[POST 요청 처리 화면]

안녕하세요, 홍길동님!

주소창(URL)에 이름은 보이지 않고, 바디에 안전하게 담아서 보냅니다

[돌아가기](#)

참조5. [그룹 채팅 앱] - 구성도

■ [그룹 채팅 앱] 구성도

HTTP
TCP
IP
LINK

프로토콜 스택

파이어베이스 구글 Cloud DB

공공데이터 포털
기상청

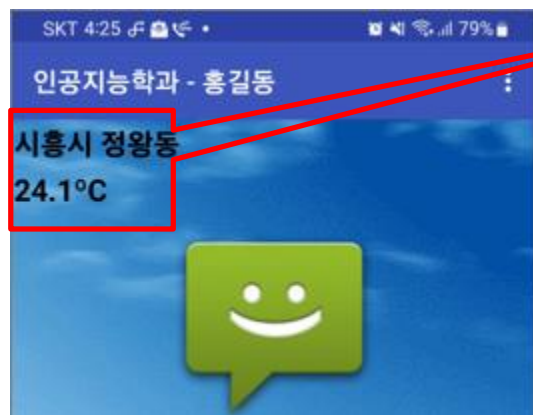
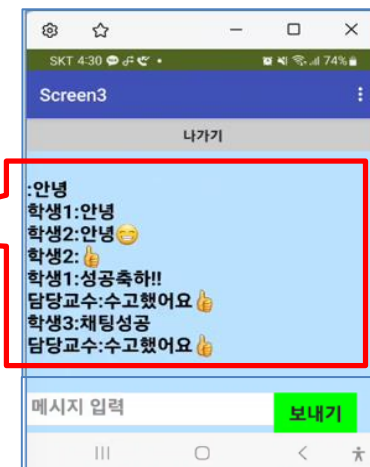
정왕동
날씨 파싱
(12주차 실습)



CDB

DB 저장하기/가져오기

웹 통신(HTTP)



참조5. [그룹 채팅 앱] - 앱 설명

■ Screen1 : 초기화면(날씨 파싱 + 로그인)

- 디자인은 자유입니다. 톱 이름도 자유입니다. 단, 제시된 기능은 정상동작해야 함

① 정왕동 날씨 파싱
(11주차 코딩블럭 재활용합니다!)



② 아이디와 비밀번호 입력 후 로그인 기능
→ Screen3 채팅화면으로 이동

GET 요청메소드
request



- 파싱(Parsing): 웹 페이지에서 원하는 데이터를 추출하여 가공하는 것

참조5. [그룹 채팅 앱] - 앱 설명

■ Screen2: 회원가입 화면

- 디자인은 자유입니다. 단, 제시된 기능은 정상동작해야 함

① 아이디와
비밀번호로 회원가입
및 Screen1으로 이동

② 취소하기 버튼
클릭 시 Screen1으로
이동

DB의 **user 버킷**에
아이디와 비밀번호 저장

파이어베이스
구글 Cloud DB
(user, chat 버킷)



chat
user
담당교수: ""1234""
손님1: ""1234""
학생1: ""1234""
학생2: ""1234""
학생3: ""1234""

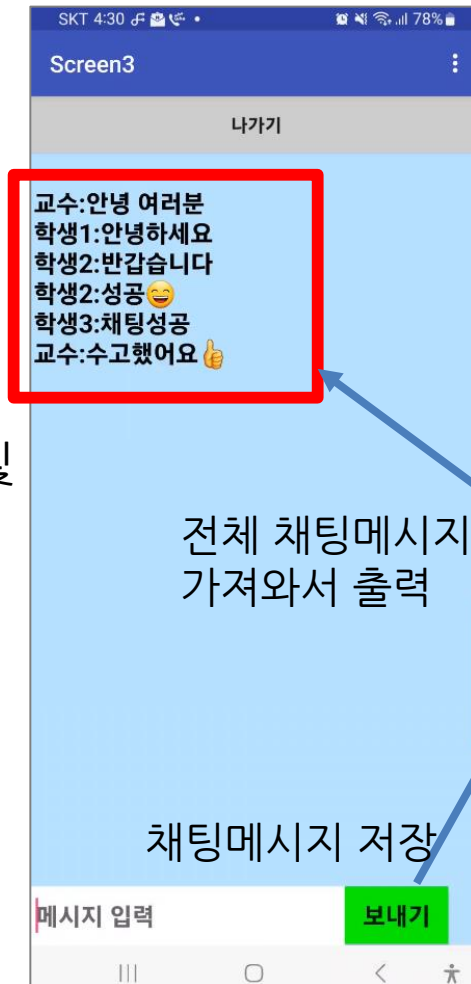
참조5. [그룹 채팅 앱] - 앱 설명

■ Screen3: 그룹 채팅 화면

- 디자인은 자유입니다. 단, 제시된 기능은 정상동작해야 함

② [나가기] 버튼
클릭 시 Screen1으로
이동

① 채팅메시지 전송 및
DB저장된 메시지
출력 기능



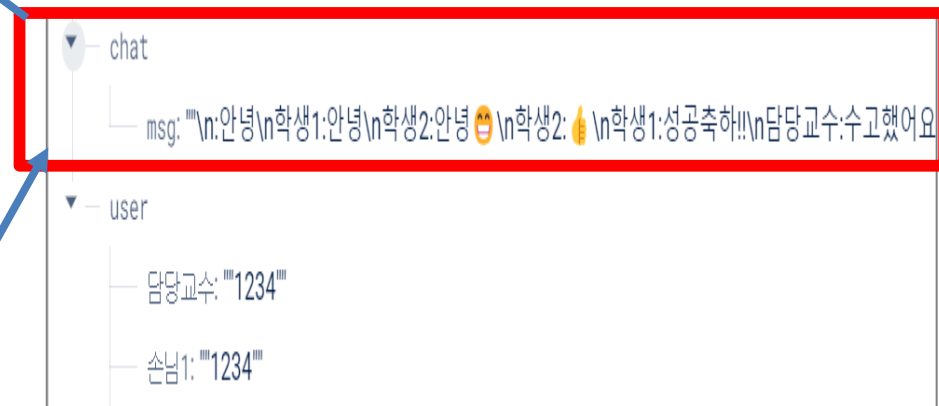
파이어베이스
구글 Cloud DB
(user, chat 버킷)



채팅 메시지는
DB의 chat 버킷에 저장

전체 채팅메시지
가져와서 출력

채팅메시지 저장



참조5. [그룹 채팅 앱] - Screen1 코딩

■ 그룹채팅 앱 제작하기 - Screen1(날씨정보 파싱) 코딩

① 기상청으로부터 날씨정보 가져와서 파싱하기 (12주차 Weather 앱 활용)



전역변수 만들기 APIkey 초기값 'R'

전역변수 만들기 weather 초기값 ''

전역변수 만들기 weather_data 초기값 ''

전역변수 만들기 weather_temp 초기값 ''

전역변수 만들기 current_date 초기값 ''

전역변수 만들기 current_time 초기값 ''

전역변수 만들기 base_date 초기값 ''

전역변수 만들기 base_time 초기값 ''

12주차 Weather 앱 코드 재활용

언제 Screen1 초기화되었을때

실행 지정하기 시계1 . 타이머활성화여부 값 참

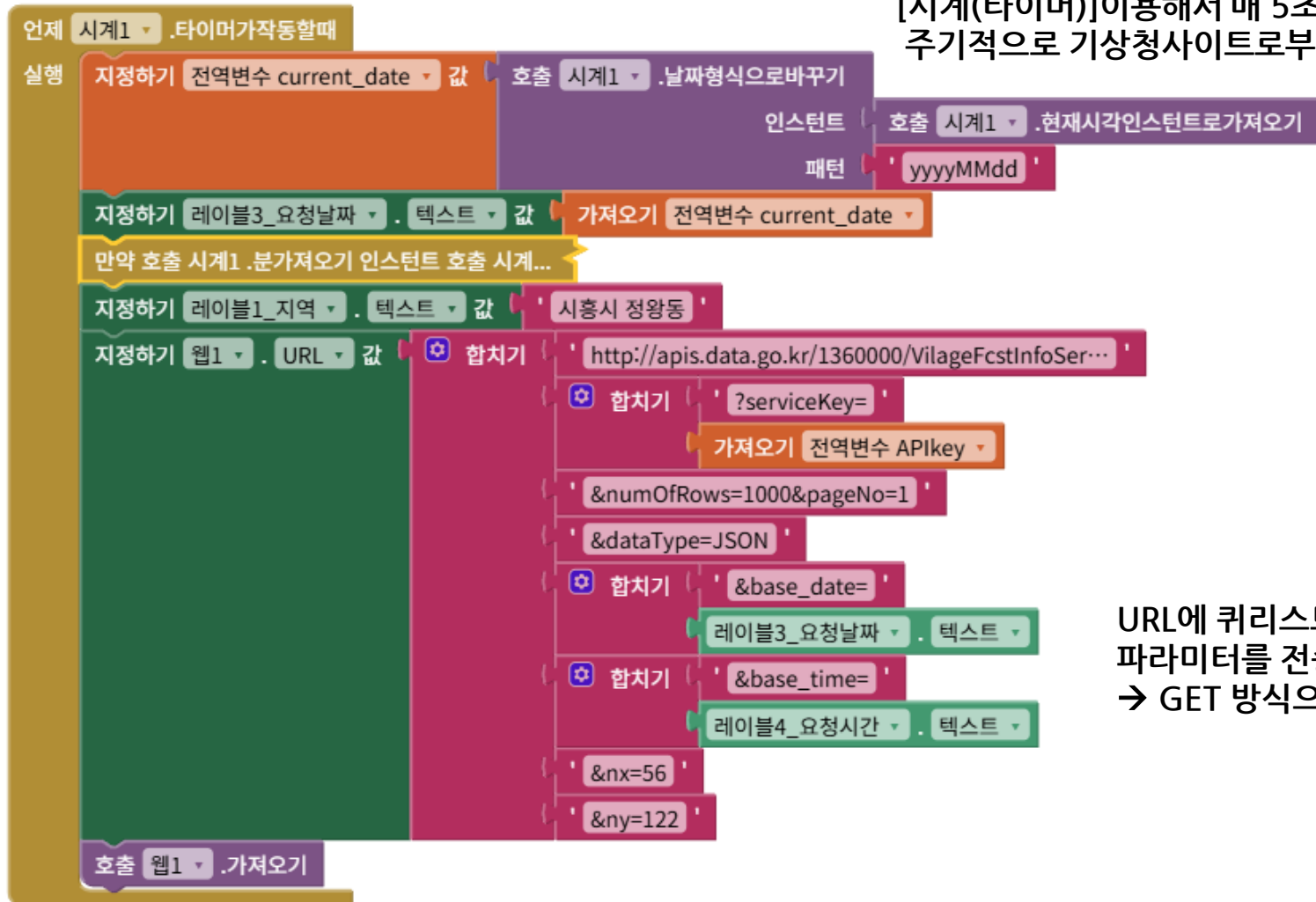
[시계(타이머)] - 타이머 주기 설정
앱이 초기화 되었을 때 5초 타이머 시작
매 5초마다 타이머 작동

*주기적으로 실행해야 하는 작업이 있을 시 사용

참조5. [그룹 채팅 앱] - Screen1 코딩

■ 그룹채팅 앱 제작하기 - Screen1(날씨정보 파싱) 코딩

① 기상청으로부터 날씨정보 가져와서 파싱하기 (12주차 Weather 앱 활용)



[시계(타이머)]이용해서 매 5초마다 타이머 작동
주기적으로 기상청사이트로부터 기온정보 가져오기

URL에 쿼리스트링으로
파라미터를 전송
→ GET 방식으로 요청

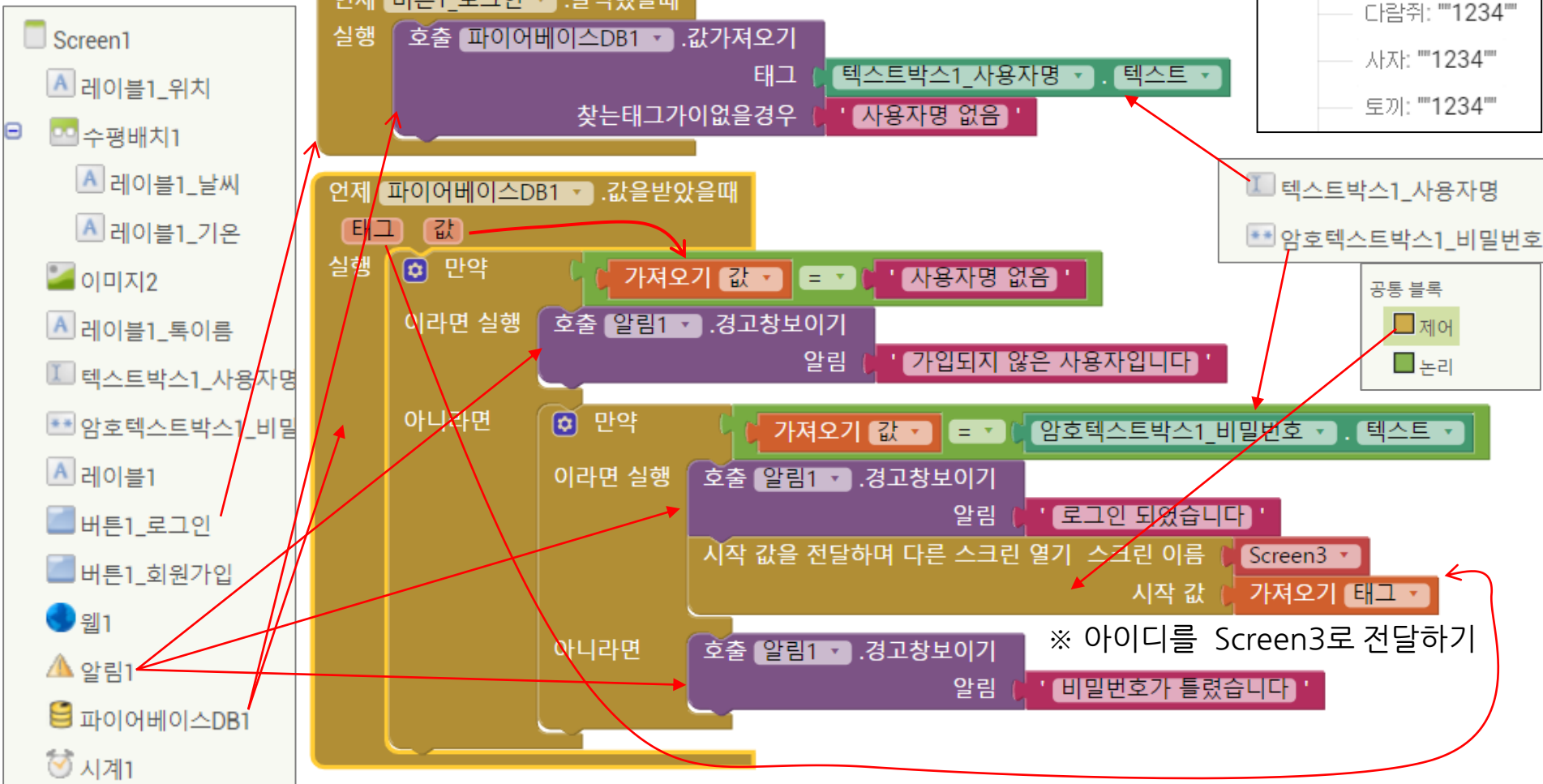
참조5. [그룹 채팅 앱] - Screen1 코딩

■ 그룹채팅 앱 제작하기 - Screen1(로그인) 코딩

③ 아이디(이름)과 비밀번호로 로그인 하기

태그(아이디): 값
예시) 홍길동: "1234"

user	
haha:	"1234"
hong:	"1234"
kim:	"1234"
lee:	"1234"
park:	"1234"
거북이:	"1234"
다람쥐:	"1234"
사자:	"1234"
토끼:	"1234"



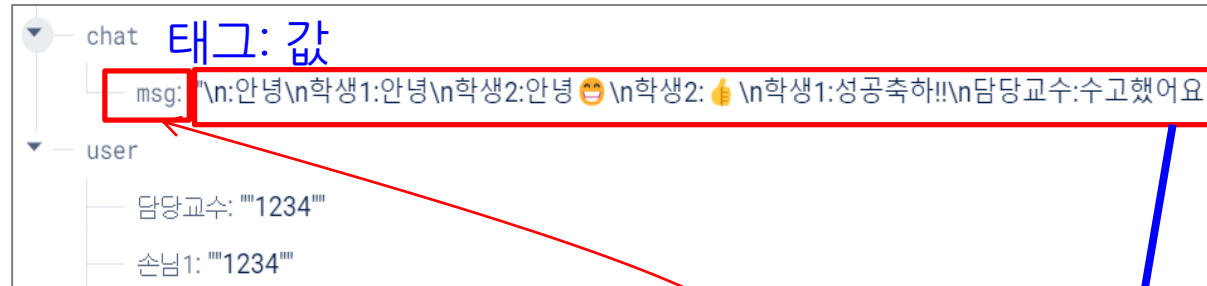
※ 아이디를 Screen3로 전달하기

참조5. [그룹 채팅 앱] - Screen3 코딩

■ 그룹채팅 앱 제작하기 - Screen3(그룹채팅) 코딩

- 주기적으로 DB에 저장된 채팅 메시지 가져와서 앱 화면에 출력

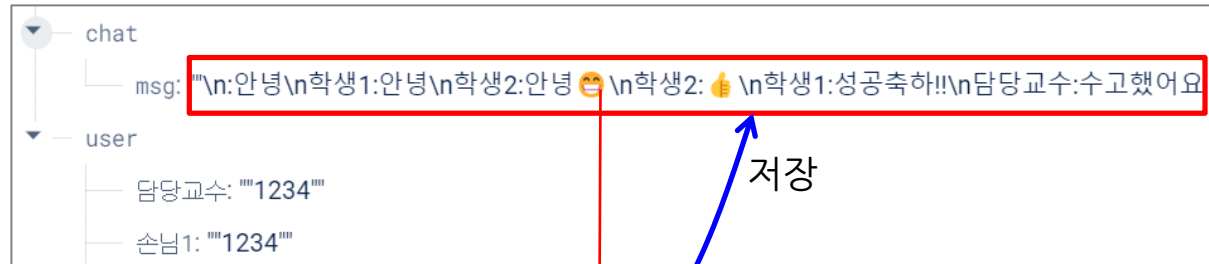
※ [시계(타이머)] 컴포넌트의 역할:
1초마다 DB에 저장된 채팅메시지
가져와서 앱에 출력하기 위해
시계(타이머) 사용



참조5. [그룹 채팅 앱] - Screen3 코딩

■ 그룹채팅 앱 제작하기 - Screen3(그룹채팅) 코딩

○ 채팅 메시지 DB에 저장하기



DB에 저장된 이전 메시지에다 신규메시지 추가하여 저장하는 방식





Thank You
